IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 22 May 2025, accepted 13 June 2025, date of publication 23 June 2025, date of current version 10 July 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3582312

RESEARCH ARTICLE

A Novel Model for Authority and Access Delegation Utilizing Self-Sovereign Identity and Verifiable Credentials

ŠPELA ČUČKO AND MUHAMED TURKANOVIĆ
Faculty of Electrical Engineering and Computer Science, University of Maribor, 2000 Maribor, Slovenia
Corresponding author: Špela Čučko (spela.cucko@um.si)

This work was supported by Slovenian Research Agency (Research Core Funding) under Grant P2-0057.

ABSTRACT Self-Sovereign Identity (SSI) is a decentralized, user-centric approach to Identity Management (IdM), that allows users to gain control over their digital identity and data. It brings many advantages and reduces the shortcomings of traditional, centralized IdM systems. Despite its positive features, SSI focuses predominantly on direct interactions between two independent entities. It enables direct identification, authentication, authorization, and access to resources and services where the identity holders are the authenticated bearers of their credentials. On the other hand, it does not address primarily indirect identity control, such as delegation, which is an essential part of life situations and natural human relationships. Delegation is particularly common in organizational contexts, and usually arises from the need for greater efficiency, relief, or distribution of work, collaboration, and cooperation and the inability to perform work, tasks, responsibilities, and duties for various reasons. However, it is relatively unexplored within the SSI framework, and requires additional research efforts and the development of new approaches. Consequently, our research aimed to reduce the aforementioned gap and address delegation in the SSI ecosystem comprehensively. Thus, we present a novel authority and access delegation model utilizing Verifiable Credentials, that enable delegation support for existing use cases, workflows, and processes of organizations and companies, providing resources and services to end users. The model includes delegation building blocks, data objects and data flow, delegation phases and processes, and technology-independent components. It is validated with a case study in the field of Education.

INDEX TERMS Access delegation, authority delegation, delegation, self-sovereign identity, verifiable credential, verifiable mandate.

I. INTRODUCTION

Self-Sovereign Identity (SSI) is a decentralized, user-centric concept representing a paradigm shift in how digital identity is perceived and managed. It presents a shift of power and control from centralized authorities, such as the Certificate Authority (CA) and Identity and Service Providers (IdPs and SPs), to end-users or identity holders within the SSI's scope in the center of digital interactions. Therefore, they are responsible for identity provisioning and management; for creating, storing, and managing their Decentralized Identifiers (DIDs) and cryptographic material; obtaining (identity) attributes in the form of Verifiable Credentials (VCs) from various sources, storing them, and transmitting them as proof of identity when required. Consequently, SSI addresses the shortcomings of traditional, centralized approaches, such as Single Point Of Failure (SPOF), centralization of power and control by IdPs and SPs, centralized data storage of Personally Identifiable Information (PII), and reliance on intermediaries to provide identity (IdPs, CAs), which can lead to personal data misuse, data leakage, identity theft, and other security and privacy-connected risks. On the other hand, it brings many advantages to the field of Identity Management (IdM), as it follows modern technological principles and approaches, such as the decentralization of power, control, and data, user-centrism, enhanced security and privacy,

The associate editor coordinating the review of this manuscript and approving it for publication was Diana Gratiela Berbecaru.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

data sovereignty, minimal or selective data disclosure, interoperability, mobile-first design, and digital wallets, that present a crucial component in the SSI ecosystem and are consistent with the efforts of the European Commission. Despite its advantages and positive features, SSI focuses predominantly on direct, one-to-one interactions between two independent entities. It facilitates direct identification, authentication, authorization, and access to resources and services where the identity holder is the authenticating bearer of their credentials. Therefore, it supports primarily cases where identity holders use and manage their digital identifiers (DID), cryptographic material, and credentials (VC). In this regard, SSI does not address Indirect Identity Control (IIC) primarily, essential to everyday life situations and natural human relationships in the physical world, such as guardianship, controllership, and delegation. Consequently, SSI excludes individuals who, due to social, legal, mental, physical, or other limitations, are unable to act independently in the digital environment. These include children, the elderly, the homeless, animals, and other entities needing proxies to manage their identity. Furthermore, current SSI models do not address situations where a delegation of tasks, rights, responsibilities, or permissions is required effectively, either due to the need for increased efficiency, relief from work, cooperation, and collaboration, or due to the temporary or permanent inability of an entity (e.g., an individual) to perform certain activities or tasks. The aforementioned situations are prevalent in an organizational context for companies and organizations where delegation plays a key role. It enables flexible management of resources, access, and responsibilities. In hierarchical structures, it is crucial for transferring access rights, making decisions, or performing tasks to subordinate entities, which allows for increased efficiency, instilling trust, reducing overload, and promoting collaboration. It is important within (among employees) and across organizational boundaries (partners). In addition, it enables security policy optimization and the expansion of Access Control (AC) mechanisms, by enabling shared or time-limited access to critical systems and data. While delegation is an essential and often necessary feature for companies and organizations, it is beneficial for their end users, as it represents added value, and allows users to empower third parties (other identity holders) to act or access services and resources on their behalf. This is especially relevant when accessing and managing health, financial, or personal data, and services when users cannot perform specific actions due to illness, travel, or other limitations. Additionally, delegation is essential in scenarios that require the cooperation of multiple entities, such as while working on projects, in situations where automated execution of tasks by intelligent agents or digital assistants is required, in temporary guardianship, and in many other situations that require a transfer of tasks, rights, responsibilities, or permissions between entities. Delegation represents a fundamental part of the real-life scenarios we often face in physical and digital environments. Therefore, support for delegation in various digital systems and applications provides a better user experience, greater flexibility, and security in managing identities and rights in the digital world.

Although some advancements have been made in defining the concepts of IIC and delegation within SSI, a comprehensive and standardized delegation model is still lacking. An ideal model would delineate the fundamental building blocks, data objects, data flows, phases, and processes of delegation, clearly, as well as the components that need to be integrated within SSI wallets/agents to facilitate various delegation scenarios—from simple task delegation to complex organizational authority hierarchies.

While it has been demonstrated that VCs can be adapted to express delegation rights, permissions, and constraints, a standardized approach has yet to be established. Moreover, several core challenges remain unresolved, including the representation and verification of delegation chains, revocation mechanisms that preserve consistency across multi-level delegations, support for auditability and transparency, and the governance of delegation policies in decentralized ecosystems. These gaps highlight the need for further research and development to enable robust, flexible, and privacy-preserving delegation mechanisms that align with the SSI principles and real-world requirements.

In summary, SSI focuses on supporting direct interactions, while use cases that require indirect identity control, such as delegation, are extremely limited and relatively unexplored, but offer interesting research opportunities. Many unsolved challenges require additional research efforts and the development of new approaches and innovative solutions that will enable secure and efficient delegation of authority and access in different contexts and domains, which is essential for the wider adoption and use of SSI, as only in this way can we achieve reliable and user-friendly IdM, which will contribute to greater security, privacy, and trust in digital services.

A. AIM, METHODOLOGY, AND CONTRIBUTION

Considering all the challenges addressed above, the aim of this research was to address the delegation process comprehensively, and demonstrate that, by utilizing SSI principles, properties, and technologies, we can:

(A1) Digitalize and automate all delegation processes, including those that are currently executed physically.

(A2) Support both the delegation of authority and the delegation access.

(A3) Provide critical functionalities derived from existing delegation requirements.

Thus, the main objective was to develop and propose a comprehensive delegation model that deals with the problems and challenges of traditional, centralized approaches, and can compete with the existing delegation models.

In order to achieve the aforementioned objectives, we employed several research methods:

(M1) **Literature review:** We conducted a literature review to examine the existing research in the fields of SSI, delegation,

and IIC, focusing primarily on delegation using the SSI approach.
(M2) **Requirements analysis:** We identified delegation requirements, which, together with the insights obtained from the literature review, provided the foundation for the model's development.
(M3) **Model development and validation:** We developed a novel authority and access delegation model encompassing multiple core elements. Last but not least, we validated the model through a case study in the educational domain, highlighting its potential to facilitate delegation across various contexts.

Consequently, the main contribution of this research is a novel model for authority or access delegation utilizing the VC data model. The novel model represents an expansion of the existing SSI model to support the delegation of authority or access in existing use cases, workflows, and (business) processes of organizations and companies that provide resource(s) and service(s) to the end users (i.e., identity holders). The model includes:
(1) Presentation of the basic **building blocks**, their interrelationship, and the general **data flow** of crucial data objects.
(2) A detailed presentation of the delegation of authority or access based on five **phases**, which include several key **processes and sub-processes** and three **perspectives**: the perspective of the delegator, the delegate, and the provider of services and resources (SP and RP).
(3) Identification and definition of **components** to support all phases and processes with technology-independent specifications.
(4) Definition of a **reference architecture** and presentation of potential integration of key components into existing SSI **wallets/agents**, or provided as external components or services.
(5) Validation of the model based on a **case study**. Validating it with application to a real use case, formalizing it through the delegation credential, i.e., a Verifiable Mandate (VM) example that can be adjusted to support various use cases, workflows, and (business) processes.

B. LIMITATIONS AND ASSUMPTIONS

As part of our research we have accepted some assumptions (A1-5) and limitations (L1-4) related to the proposed delegation model, which are presented below and summarized in Tables 1 and 2.

Due to the complexity of the IIC space, which distinguishes between three types of entity relationships, namely, guardianship, controllership, and delegation, (L1) we limited ourselves only to **delegation**, which, in itself, brings a lot of intricacy due to its features and characteristics [1], [2]. It can take many forms, varying in the types of entities involved, such as human-to-human, human-to-machine, machine-to-human, and machine-to-machine [2]. It can be simple and direct, including only a delegator and delegate, or more complex and indirect, involving one or multiple intermediaries (sub-delegation), introducing the concept of chaining direct delegation transactions, which allows greater scalability and flexibility, but brings additional considerations [1]. Moreover, we can see the demarcation between access and authority delegation [3]. The former focuses mainly on delegating permissions or rights to access specific resources or information. In contrast, the latter refers to transferring power over decision-making or control of operations/actions to another entity. Within the framework of our research, (L2) we focused mainly on **direct delegation** in **human-to-human** interactions. However, the proposed model can also support more complex delegation use cases involving intermediaries and other forms of delegation, such as human-to-machine, etc.

(L3) We also limited ourselves to the **SSI approach**, utilizing SSI principles and properties, highlighting the importance of decentralization, autonomy, user-centrism, control over data and interactions, privacy, data minimization, and security, among others, presented in [4]. In addition to the above concepts, SSI is consistent with the mobile-first approach, it emphasizes the importance of GDPR, and it is also aligned with the efforts of the European Commission [5], which aims to provide digital identity wallets enabling digital identification for every citizen, resident, and business within the EU. Thus, (A1) we assume the utilization of basic SSI building blocks to extend the SSI model and support delegation.

Within the SSI ecosystem, formal delegation can be achieved through a DID document (DDO) and VC delegation. The first, typically, focuses on identity control capabilities and DID governance, while the second enables entities to delegate roles, rights, permissions, or claims, providing more flexible and granular delegation. Hence, (L4) we limited ourselves to the **VC delegation**, which, unlike the DID delegation that can be done in a standardized way utilizing the `capabilityDelegation` property in DDO [6], is not yet standardized, and requires additional research efforts. Therefore, the proposed model utilizes VC, and extends the basic VC data model specified and standardized by the World Wide Web Consortium (W3C) [7], [8] to support delegation. From the SSI trust triangle perspective, we can distinguish between delegation by (1) issuers and verifiers (usually Legal Entities (LEs)), and (2) identity holders (usually a Natural Person (NP)). The first is focused primarily on delegating the VC issuance or verification to another entity that carries decision-making powers, and is, thus, more aligned with the definition of authority delegation. On the other hand, the second aligns more with the definition of access delegation, as it is focused on delegating permissions to access (and performing some actions on specific resources or services). As such, **delegation by identity holders** is at the heart of our research, which means that one identity holder (delegator) can transfer (some of) its powers to another identity holder (delegate). (A2) Whereby we assume that the delegator and delegate are either LEs or NPs, while the verifier is an LE that integrates/supports delegation within its

TABLE 1. Review of the limitations of the proposed delegation model.

ID	Limitations
L1	Delegation, as part of the IIC.
L2	Direct delegation in human-to-human interactions in real-life scenarios.
L3	Utilizing the SSI approach; SSI principles and properties.
L4	VC as a means for delegation.

TABLE 2. Review of the assumptions of the proposed delegation model.

ID	Assumptions
A1	Utilization of the basic SSI building blocks.
A2	Involvement of a delegator, delegate (LE or NP), and verifier (LE).
A3	Utilization in the public or private sector in digital and physical environments.
A4	Expansion of existing companies/organizations use cases, workflows, and processes of companies/organizations.
A5	Support for revocation and suspension.

use cases, workflows, or (business) processes. The latter is imperative, as delegation requires setting up the infrastructure and managing the process, which is more difficult for NPs, as it requires more resources. (A3) In addition to the above, we assume the utilization of the model in both digital and physical environments in the public or private sector, thus not limiting ourselves in that regard. (A4) Moreover, as is evident from the above statement, we assume the use of the proposed model to expand existing use cases, workflows, and (business) processes of companies/organizations (both within and cross-domain). By this, we meant that permissions/grants/actions can be delegated between entities that are part of the company/organization (e.g., employees) as well as between external entities that do not belong to the domain (e.g., end users of resources or services provided by the company/organization).

Last but not least, as revoking and/or suspending delegation is crucial for maintaining the delegator's control and preventing misuse and abuse, especially in changed circumstances, (A5) we assume the VM status change is enabled by utilizing one of the suggested revocation mechanisms that allow the VM issuer (and, in some cases, VM verifiers), to change the status of the VM. However, status change (revocation, suspension, etc.), is beyond the scope of this research, as it represents a broader research area of VC management, which has already been addressed in many studies [9], [10], [11], [12], [13], [14], [15] that seek to develop a revocation mechanism consistent with the principles of SSI.

The remainder of this paper is structured as follows. In the upcoming Section II, we present the research background, highlighting important concepts and related works that are crucial for understanding the SSI approach, IIC, and delegation. This is followed by Section III, in which we present a novel authority and access delegation model utilizing VCs striving to achieve SSI principles and properties. The model represents an extension of SSI, and includes several elements elaborated and presented in more detail in the subsections. Section III-A presents the model's basic building blocks, and Section III-B shows their interconnection, presenting basic data flow and highlighting the essential data objects. Section III-C outlines the overall generalized delegation process, consisting of five essential phases for implementing the entire delegation process successfully and efficiently. Section III-D identifies and describes key delegation-specific components, whose inclusion in wallets/agents is discussed in Section III-E, where it is addressed from the perspective of the involved entities. The model is validated with a case study in the educational domain (Section IV). In addition to descriptions and analysis, this section also includes an example of a VM credential (Section IV-A4), which can be adapted for different use cases and their requirements. The penultimate section (Section V) discusses the proposed model's benefits and advantages, linking them to a validated case study for illustrative purposes. Last but not least, Section VI presents a conclusion and opportunities for further research.

II. PRELIMINARIES AND RELATED WORKS

A. SELF-SOVEREIGN IDENTITY

Self-Sovereign Identity (SSI) is a user-centric, decentralized approach to identity management [16], [17] that enables entities, i.e., identity holders, to control their digital identities (identifiers and identity data) fully without relying on any external, central authority for identity provisioning, management, and storage. Thus, it strives to minimize or even eliminate entirely, reliance on intermediaries acting as CAs or IdPs in digital interactions, tackling the challenges and concerns of traditional, centralized approaches, such as SPOF, centralized control and data storage, vendor lock-in, personal data misuse, connected to data breaches, leaks, and identity thefts, as well as other privacy and security threats [18], [19]. It empowers entities to be in control of their identity and data flow during digital interactions, and presents a means for digital identification, authentication, and authorization. It allows entities to (1) create and manage an unlimited number of unique Decentralized Identifiers (DIDs) for different interactions independently of any external entity, i.e., centralized authority [20], (2) establish relationships (i.e., peer-to-peer connections) with different stakeholders (i.e., service and resource providers, SP and RP, providing value to the end users) within the SSI ecosystem, (3) obtain identity data/attributes/claims in the form of Verifiable Credentials (VCs) from various sources (i.e., third-party credential issuers), (4) store identifiers, cryptographic material, and obtained data/attributes/claims in digital identity wallets or off-chain personal repositories of their choice securely and autonomously, (5) present/share the

obtained data/attributes/claims privately and securely when proof of identity is required, while remaining in complete control of the identifiers and associated data.

This can be achieved by utilizing basic SSI building blocks that will be explored in more detail in Section III-A, namely, a trust triangle, Decentralized Identifiers (DIDs), Verifiable Credentials (VCs), and Presentations (VPs), schemas, digital wallets and agents, off-chain data storage, Verifiable Data Registries (VDRs), and governance, trust frameworks. These building blocks are crucial for an effective SSI ecosystem, and also for establishing delegation processes within it.

B. DELEGATION

The main focus of this paper is delegation as part of IIC, as defined by the Sovrin Foundation [21], [22]. In this context, delegation is the process in which one identity owner, i.e., the delegator, delegates (part of) its work, tasks, rights, permissions, responsibilities, or capabilities to another identity owner, i.e., the delegate, who can then act on behalf of the delegator. This means that the delegator authorizes or empowers the delegate to act on its behalf.

Within the digital environment and the framework of traditional delegation, we can distinguish between different forms of interaction, which can be divided into human-to-human, human-to-machine, machine-to-human, and machine-to-machine delegation. The most commonly used interaction is human-to-human [2], which is also the focus of this research. An example of such an interaction is when a professor delegates some task to an assistant, or when a student authorizes his/her parents to enroll him/her (on his/her behalf), in a particular study program at the university. An example of delegation, based on human-to-machine interaction, is when a user delegates some of his/her rights to a web application [2], as in the case of the OAuth protocol [23]. Moreover, in the context of SSI, the representation of such interaction is when an identity holder delegates some work to various agents that can (partly) automate processes and SSI flows, which can improve the User Experience (UX). Machine-to-machine delegation is based on the idea that one machine transfers some of its rights to another machine in a predefined and (semi)automated way, which is a common scenario in the Internet of Things (IoT) context.

Within the SSI, there are three possible approaches to delegation, namely, delegation (1) by expressing intent in a DID document, (2) with VCs, and (3) by sharing a digital identity wallet [24]. All the methods are feasible, but have different advantages, disadvantages, and risk levels, and can meet different delegation requirements. It has been shown that formal delegation is achieved best by utilizing VCs, which is also the focus of this research. This approach allows delegators to create a special type of VC, that specifies the delegation scope, and defines permissions/authorizations and restrictions/constraints for each relationship separately. In addition, recursive delegation [25], i.e., sub-delegation, can be supported, and an audit trail can be provided. Moreover, we can take advantage of standard verification and revocation processes already established within the SSI ecosystem [24]. Therefore, delegators create and issue a VC, establishing delegation relationships and authorizing delegates by specifying the delegation scope in terms of permissions and constraints. Afterward, they forward the VCs to the delegates, who can present them to the verifiers when proof of delegation is required [26].

If we focus on the life cycle of VCs and the roles that entities can play within the SSI ecosystem, we can distinguish between delegation by holders, issuers, and verifiers [27]. Delegation by the holder is the most common in various real-life situations, as it refers to the end user. It allows the identity holder to transfer the obtained VC to another identity holder who can assert a VC for itself (in the case of a bearer credential) or on behalf of the original holder. Consequently, it addresses situations where one identity holder wants to delegate part of their work, tasks, rights, responsibilities, or capabilities to another identity holder who can then act on its behalf [27], for example, to enroll into first-cycle undergraduate study programs at a higher education institution (as demonstrated in Section IV-A), when the holder cannot perform this action by himself/herself. Delegation by the issuer or verifier increases the scalability of VCs, as it focuses on delegating the ability to issue or verify VCs, empowering multiple entities to become issuers or verifiers in the SSI ecosystem. Delegation by the issuer focuses on transferring the authority to issue VCs to another identity holder, who becomes the VC issuer. Two delegation mechanisms can be used in this regard, namely, downwards and upwards delegation. The first mechanism allows multiple independent entities to issue the same type of VC. The issuance of VCs is, thus, not limited to just one issuer, to the Source of Authority (SoA), but is dispersed among multiple entities that become subordinate issuers. For example, Visa Inc. transfers the issuance of credit cards to its partner banks, or a university accredits each of its Faculties to issue diploma VCs. The second mechanism is the opposite, as the issuance of VCs is transferred from subordinates to superiors or parent entities to SoA in each domain. For example, National Student Membership Cards are issued by the National Union of Students rather than by each university. Delegation by the verifier focuses on transferring the authority to verify VCs to another identity holder, who becomes the VC verifier. It allows verifiers to delegate the validation of VCs and the authorization of holders to numerous external or third-party entities, who then verify the VCs on their delegators' behalf. The result of the verification process can then be sent to the delegator [27].

C. RELATED WORKS

The VC data model [7], [8] is a Standard that defines how we can issue, exchange, and verify digital credentials securely and privately. It allows individuals, organizations, and subjects to prove specific properties or facts about themselves

in a cryptographically verifiable way. The specification [28] distinguishes between a VC subject and a VC holder, as the two entities are not necessarily the same (although, in most cases, they are). Thus, it defines various possible subject-holder relationships that can be expressed by the VC data model, including a relationship where the holder is not the subject of a VC credential, reflecting real-world use cases, and supporting circumstances in which the VC subject is unable or unwilling to control or manage (parts of) its identity due to medical, social, legal, organizational, or other reasons. When a VC identifies a subject that is not the same as the holder uniquely, then (1) the subject may have forwarded the VC to the holder (e.g., power of attorney, employee), (2) the holder may have been authorized by the issuer independently (e.g., law enforcement), (3) the holder may be acting on behalf of the subject (e.g., pet owner), (4) the holder is acting for the verifier, or (5) it is a random holder. The specification [28] includes several suggestions on how to handle such relationships. Namely: (1) the subject of VC_1 can issue a new VC_2, which includes a copy of the VC_1 issued by the (trusted) issuer, and transfer both VCs to the holder, (2) the subject of VC_1 can express its relationship with the holder within the new VC_2 and forward both VCs to the holder, (3) the issuer may, in addition to VC_1, describing the subject, issue a new VC_2 expressing the relationship between the subject and the holder, (4) the issuer may also express the relationship between the subject and the holder in the VC_1. If there is no relationship between the subject and the holder, the issuer can indicate its relationship with the holder (3, 4) and send the VCs directly to the holder. Although the delegation is outside the scope of the VC data model specification, the model can be used and adapted for delegation purposes.

Lim et al. [29] deal indirectly with the VC delegation. In their work, they focus on the holder-centric structure of VCs, and thus address the challenge of having holders who can transfer credentials without the subject's permission. They proposed a subject-centric VC, in which the use of a VC by a holder, presenting VC as a proxy, requires the involvement of the VC subject. This approach addresses a key limitation of the W3C VC data model, which does not consider the potential loss of control over VCs, or the unauthorized secondary transfer of credentials, thereby making VCs inherently holder-centric. To mitigate this, the authors suggest encrypting credentials with the subject's public key before VC transfer, ensuring its content remains inaccessible without the subject's explicit permission. The holder generates a VP without decrypting the credential and presents it to a verifier, who then requests credential decryption from the VC subject, who can either grant or deny the request. This approach limits the use of the original VC, and ensures that the subject remains involved actively, thereby effectively complementing the VP verification process. However, the approach has some limitations. It is not fully compatible with selective disclosure techniques, lacks mechanisms for tracking credential transfers, and is unsuitable for scenarios involving multiple subjects within a single VC.

The Sovrin Foundation states that SSI, as a new digital identity model, brings many advantages, but is limited directly to use by individuals who have digital access, i.e., appropriate technology and access to the internet, and legal, mental, and physical capacity to act independently, both on and off-line [30]. In their white paper [22], [31], they address the limitation above by exploring the concept of guardianship, and highlight the importance of expanding the SSI model by introducing individuals and organizations that serve as digital guardians and enabling digital inclusion for all. They (i) define the concept, (ii) identify its various use cases, thereby emphasizing the necessity of supporting the broader adoption of SSI, (iii) explore the dynamic, contextual nature of the relationships between the involved entities, and address the impact of temporal and self-sovereign dimensions, (iv) analyze the guardianship life cycle, from inception to termination, (v) identify risks and vulnerabilities, and provide examples for their mitigation, and (vi) demonstrate its integration within the four layers' architecture in the Trust over IP (ToIP) stack [32], [33], [34]. In addition to guardianship (the guardian represents the dependent), they (vii) identify two additional types of IIC, in which identity holders exert control on behalf of the subject: delegation (a delegate acts on behalf of a delegator), and controllership (a thing controller controls a thing). They define all types of IIC, and present their use cases, commonalities, differences, and interconnections [26], which can lead to transition from one form of relationship to another. For example, a guardian may delegate part of its rights and responsibilities to another entity during its absence, wherein this delegation can become a controllership if it involves the management of a thing, etc. The white paper is accompanied by two other documents, describing the technical requirements [35] and the implementation [36] of the guardianship credential. Due to the common characteristics and close interrelation between the mentioned types of IIC, Sovrin's contribution is crucial. It represents the fundamental step in the field of IIC within the framework of the SSI ecosystem. Despite Sovrin's focus on guardianship, the results of their research can serve as a basis for understanding the concept and transferring existing knowledge to other forms of IIC, which Sovrin did not address in detail. They can influence the development of conceptual models, the identification of requirements, and the implementation of the remaining two types of IIC, i.e., delegation and controllership significantly.

Based on Sovirn's findings, the Hyperledge Aries RFC 0103 [21] proposes a mechanism for implementing IIC that allows flexible, precise, and secure modeling of each IIC relationship. Regardless of the IIC type, the solution includes (1) a proxy trust framework, (2) a proxy credential, and (3) a proxy challenge (where the term proxy is being used generically, and could be adjusted as required, e.g.,

delegation framework/credential/challenge). (1) The proxy trust framework represents a human and machine-readable document or collection of documents, that specify the rules and conventions that serve as a guideline for a particular type of use case. The framework is crucial for both proxy credential creation and proxy challenge execution. (2) The proxy credential describes the controlled entity and its proxy, specifies their relationship, and defines the scope of powers (roles, permissions, rights, duties) obtained by the proxy precisely. It may also include restrictions of control over the specific relationship (constraints and limitations), a link to the trust framework, and optional links to an auditing, appeal, and revocation service. The credential must comply with the W3C VC data model conventions, and can utilize any style of proof or data format (JSON-LD, JWT, Sovrin ZKP, etc.). (3) The proxy challenge serves as a means for verifying and evaluating a proxy credential according to its presented context. It requires the proxy to create a VP based on the proxy credentials, and, in some cases, requires the inclusion of additional VCs serving as proof of identity. This is followed by credential evaluation, which includes traditional verification, comparison of the proxy's role to permissions, and comparison of the circumstances to constraints. It provides evidence for exerting indirect control, and creates an opportunity for auditing. The provided technical solution is explored through a guardianship use case, precisely, through an orphan/refugee example. It includes a sample schema [37] for issuing guardianship credentials and a sample guardianship trust framework [38], demonstrating their structure and content. As they believe all IIC forms should use similarly structured credentials in similar ways in the context of similarly structured trust frameworks, the proposed solution could be used to implement all IIC types, including delegation, with adjustments reflecting the requirements of the individual use case.

Dündar and Sertkaya [39] addressed the IIC, focusing on custodianship as a subset of guardianship. The authors presented a use case for joint custody, demanding the involvement of two guardians, who must manage the dependent's identity collaboratively. This common real-life situation often arises from divorce. They implemented the Proof of Concept (PoC), including custody trust framework and credential schema, that was based on prior solutions introduced by the Sovrin Foundation and the Hyperledger Aries. They concluded that the proposed framework could be integrated into existing SSI systems seamlessly without causing any overload.

RFC 0104 [25] proposes credential chaining, to achieve data provenance while maintaining verifiability and altering trust dynamics within the SSI ecosystem. The RFC introduces the so-called Chained Credentials (CC), that leverage the W3C VC data model, and encapsulate information about the provenance of all or part of the data employed within the credential `provenanceProof` property, which enables traceability of data back to their origin. Therefore, for each claim from the `credentialSubject` property, the `provenanceProof` property can embed a W3C VP, that proves the origin of the claim's value. CC can be combined with RFC 0103 [21], to support more complex use cases and achieve sub-delegation of authority and privileges among multiple entities (intermediaries), enabling the establishment of a data provenance chain. Each credential in the chain acts as a capability token, granting the CC holder privileges derived from the upstream issuer's authority. CCs are especially useful in the context of a multi-level hierarchy of organizations/companies when superiors delegate to direct subordinates, and in use cases where intermediaries in the chain are not globally trusted or legal entities (LEs), which is a very common, if not the dominant, scenario in the case of delegation. Therefore, CCs can serve as a mechanism for building trust and supporting so-called private issuers that do not have a globally recognized reputation, public DIDs, and VC definitions in a publicly available registry. This is possible because the trust is based on an unbroken chain and a root issuer/attester, usually an authority configured for traditional credential issuance, with a public DID and reputation. In addition to introducing credential chaining and proposing CC as a solution for data provenance, the RFC [25] demonstrates chaining importance within the IIC ecosystem with various use cases. It presents a possible solution, focusing on the chain of provenance for authority delegation, in which the participating entities transfer a subset of their permissions to operate and drive a car to another entity. The provided credentials examples combine CC and proxy credentials, exemplifying their complementarity.

Mukta et al. [40], [41] proposed a high-level credential-based CredTrust framework for trust management that includes the concept of Supporting Credentials (SCs) to establish the 'Web of Trust.' Similarly to CCs the proposed solution enables personal or private issuers who do not need a globally recognizable reputation [25], as SC specifies the issuer's trust in the holder of the SC. This enables the creation of a credential chain, where the holder of the SC subsequently becomes the issuer of the next credential in the chain. However, both approaches require a well-known, trusted issuer at the root of the credential chain [40].

Terzi et al. [42] proposed a decentralized IdM solution, leveraging SSI, blockchain, and a smart contract, in which vehicle owners (delegators) can delegate certain rights, privileges, or permissions to other entities (delegates), such as drivers or SPs, without relying on a centralized trust authority. The solution requires each entity to have its own identity and obtain an IDVC from the Registration Authority. The entities must first establish a pairwise DID connection. Afterward, the delegate can forward its IDVC to the delegator, and request some rights that can be accepted or denied. Upon acceptance, the transaction is recorded on the blockchain, along with a unique identifier known to the involved entities. It can serve as evidence to determine accountability in cases like accidents or misuse. Hence, blockchain and smart contracts

are important for handling transactions and tracking current delegated rights. The return flow includes an additional return request by the delegate and verification by the delegator.

All of the above efforts present a good starting point, but require additional research, as the proposed solutions are still in the study phase, and, for the time being, most do not list any implementations of PoCs that would prove their feasibility.

III. MODEL

The novel authority and access delegation model *(shortly delegation model)* leverages SSI's principles and properties and utilizes a W3C VC data model [7], [8] as a means for delegating authority or access to another entity. The model includes basic SSI building blocks adjusted for a delegation use case (considering the limitations and assumptions described above in Section I). It outlines the data flow, and presents the key phases and processes of delegation in the SSI ecosystem. Moreover, it contains crucial components that should be integrated within the SSI wallet/agent, or be available as external components/services to which the SSI wallet/agent can connect.

Thus, as summarized in Table 3, the model includes basic building blocks (Section III-A), data flow (Section III-B), delegation phases (Section III-C), and technology-independent components (Sections III-D and III-E).

A. BUILDING BLOCKS

In the following we present the basic concepts and, simultaneously, the essential building blocks of SSI, as our model extends the existing SSI ecosystem to support authority and access delegation. The building blocks will be presented through the lens of delegation, which requires certain adaptions that will be elaborated further in the coming sections. The building blocks, presented in detail as follows, are each key to supporting the delegation process within the SSI ecosystem.

1) ENTITIES AND TRUST TRIANGLE

The trust triangle represents the foundation of trust within the SSI ecosystem, and refers to the relationship between the three essential entities with distinct roles, namely, credential **issuers**, **holders**, and **verifiers**. Issuers create, sign, and issue credentials to holders, who receive, store, and manage them and present them to verifiers, who validate and verify the presented credentials to ensure their integrity and authenticity. In the context of delegation, the issuer of a delegation credential is the **delegator**, the holder is the **delegate**, and the verifier is the **service or resource provider, SP or RP**, an entity that enables the delegation for various use cases, workflows, or (business) processes (use case/workflow/process owner), and is thus responsible for delegation setup and maintenance. Each entity has specific responsibilities and duties, participates in different parts of the delegation process, and uses different components, which will be described in more detail in the following sections.

Delegator or VM issuer (also *represented, mandator, proxied*) delegates (part of) its tasks/responsibilities to another entity (delegate). It authorizes the delegate to act or access resource/service(s) on its behalf, generates and issues delegation credentials, i.e., VM credential (to delegate) based on a schema, defines delegation terms, including permissions, constraints/restrictions/limitations, and conditions, and specifies the delegation relationship (through delegation credential). **Delegate or VM holder** (also *representative, mandate, or proxy*) is authorized/empowered by the delegator to perform the delegated authority or access some resources. It acts or accesses resource/service(s) on behalf of the delegator, receives a delegation credential, stores it in the storage of its choice, and uses it as delegation proof when required. **VM verifier** (also *delegation enabler, service/resource*

TABLE 3. Structure of the proposed delegation model.

Building blocks	Delegation data flow, phases and process	Delegation-specific components and wallets/agents
Describing essential delegation building blocks (A1—8).	Outlining the connection between building blocks through the delegation data flow, illustrating data exchange between the involved entities and components. Outlining the delegation process and describing the key phases (C1—5) of delegation.	Presenting the technology-independent components (D1—7) required to support the delegation over all phases and processes.
Section III-A (pp. 8 - 11)	Section III-B (pp. 11 - 12) and Section III-C (pp. 12 - 15)	Section III-D (pp. 15 - 19) and Section III-E (p. 19)
A1 Entities and trust triangle (p. 8) A2 Decentralized Identifiers (p. 8) A3 Verifiable Credentials (p. 9) A4 Schemas (p. 9) A5 Verifiable Presentations (p. 10) A6 Wallets, agents, off-chain data storage (p. 10) A7 Verifiable Data Registries (p. 10) A8 Governance frameworks (p. 11)	C1 Setup and maintenance (p. 12) C2 Credential definition and transfer (p. 13) C3 Credential utilization (p. 14) C4 Validation and verification (p. 14) C5 Credential status change (p. 15)	D1 VM Schema Generator (p. 16) D2 VM Credential Generator (p. 16) D3 VM Credential Manager (p. 16) D4 VM Credential Validator (p. 17) D5 Verification Engine (p. 17) D6 Policy Generator (p. 18) D7 Policy Engine (p. 18)

provider (SP/RP)) is usually an LE, i.e., a service/resource provider or public institution that integrates delegation within existing use cases, workflows or processes. It defines the delegation credential schemas and policies, through which it defines the delegation terms, including permissions, constraints/restrictions/limitations, and conditions that reflect the use case/workflow/process requirements. It is responsible for deploying the required registries and delegation-specific infrastructure. It obtains, processes, validates, and verifies the received delegation credentials, and allows or denies delegated authority or access.

2) DECENTRALIZED IDENTIFIERS

A Decentralized Identifier (DID) is a globally unique and persistent identifier that enables decentralized, cryptographically verifiable digital identity. Each DID is formatted as a URI (`did:<DID method>:<method-specific identifier>`) that associates the DID subject (usually the controller of DID) with a DID document (DDO), which describes the cryptographic public keys, service endpoints, authentication methods, and other metadata [43]. It can also contain data from other entities, as the DID subject does not need to be the same entity as the DID controller. Moreover, a DID can have multiple controllers and delegates if the DID controller wants to delegate cryptographic capabilities to another entity. Therefore, as mentioned, we can achieve DID delegation in a standardized way utilizing the `capabilityDelegation` property in DDO.

The DID functions independently of any central authority, centralized registries, or identity providers, allowing for autonomous registration, updates, resolution, and revocation through DID specific methods. This independence enables secure, private connections between entities, each of which can create, own, control fully, and manage multiple DIDs for various digital interactions, preventing data correlation [6].

Each entity in the delegation process must be onboarded in the SSI ecosystem and create at least one DID. Whereby LEs that require formal accreditations must use public, DID methods that must be registered in a publicly available VDR (DID Registry, Public Institution Registry), while NPs should use privacy-preserving and GDPR-compliant DID methods, such as `did:key`, that are private, and not rooted in any VDR. In the context of delegation, VM verifiers are usually LEs, while delegators and delegates can be LEs or NPs. In case the delegators are NPs using private DIDs, we deal with personal VM issuers, which brings additional challenges with identification and VM verification that will be addressed later.

DIDs are important for authentication and authorization, establishing secure connections, and exchanging data between the involved entities. Peer-to-peer DID connections must be established before any interactions and data exchange. Hence, in the delegation process, such connections must be established (i) prior to VM issuance, so the VM can be issued to the desired delegate, and (ii) prior to presenting the VM to the verifier.

3) VERIFIABLE CREDENTIALS

A Verifiable Credential (VC) is the standardized data model for secure, verifiable, digital, machine-readable credentials as defined by the W3C. It includes metadata (issuer, type, limitations, etc.), identification details, assertions about the credential subject, and verification and revocation mechanisms. Basically, it is a digitally signed document that asserts claims about a subject (e.g., an individual, an organization, or a thing) that is protected cryptographically and can be verified for authenticity and integrity [7], [8]. We can differentiate between multiple VC types [44], namely, **Verifiable Attestation (VA)**, **Verifiable Mandate (VM)**, **Verifiable ID (VID)**, **Verifiable Accreditation (VAcc)**, and **Verifiable Authorization (VAuthz)**, enhancing the functionality and applicability of SSI systems. Each type can serve a unique purpose and requirements, and be used in different scenarios to facilitate trust and interoperability across digital interactions [45].

In the context of delegation, the **VM credential** (delegation credential) extends the W3C VC data model by including properties the describing delegator(s) and delegate(s), delegation terms and conditions, such as permissions/grants/actions data, constraints/restrictions/limitations, and delegation relationships between the involved entities. The VM serves as proof that the delegate can act or access service/resource(s) and perform delegated actions/operations on behalf of the delegator, and it must adhere to the relevant VM schema and Standards. If required by the VM verifier, the VM can also incorporate additional delegators' credentials, which serve to prove identity or other identity attributes. For identification purposes, VM verifiers may request additional VID credentials from NPs whose DIDs and identity information are not registered in a publicly available repository (DID Registry, Public Institution Registry). This means that delegators who are, i.e., personal VM issuers, may need to include their VID within the VM, and delegates may need to present their VID along with the VM obtained from the delegator. Moreover, the VM verifiers may also request additional VA (or VAcc or VAuthz) credentials, to prove other attributes and aspects of the delegators' and delegates' identities, such as certifications, authorizations, etc. For example, the delegator should include a diploma credential within the VM, proving that he/she meets the university's admission conditions. In other words, the VM should encapsulate credentials that the delegator would have to submit, even if he/she was acting by himself/herself. From the delegator's perspective, such credentials prove that the delegator is authorized to perform actions or access service/resource(s) he/she wants to delegate further.

4) SCHEMAS

Although schemas refer to different data objects (e.g., DIDs, VCs, etc.) and types (e.g., data verification schemas, data encoding schemas), ensuring semantic and technical interoperability as well as verifiability and extensibility, here, we highlight the credential schemas that are crucial in the

suggested delegation approach, because, as mentioned, our model focuses on VC and not on DID delegation. VCs are generated and verified based on predefined schemas that are standardized structures defining the format and semantics of the data contained in VCs. Schemas ensure that credentials are interoperable, and can be understood and processed by different systems [7], [8], [46]. They are usually defined by the VC issuers, and are stored in a publicly available trusted Schema Registry that facilitates digital interactions and data exchange, ensures quality data objects and alignment within domain-specific requirements, as well as schema reusability, and, thus, minimization of a schema explosion [45].

In the case of extending use cases, workflows, or (business) processes for delegation support within an organization or company, VM schemas must be defined and registered by use case/workflow/process owners that enable the delegation, usually by VM verifiers, or entities that define schemas and (legal) rules for individual domains. VM schemas must be registered in the Schema Registry prior to the VM issuance and verification. Hence, the VM issuers (delegators) must be able to obtain the appropriate VM schema from the registry, and generate a VM credential based on the schema. Moreover, the VM must reference the schema in the `credentialSchema` property, allowing the verifiers to determine whether the VM and its content conform to the schema [7], [8].

5) VERIFIABLE PRESENTATIONS

A Verifiable Presentation (VP) is a packaged set of one or more VCs, along with proofs, such as digital signatures, that can be shared with verifiers to prove specific aspects of an entity's identity. VPs allow holders to disclose specific information from their credentials to verifiers selectively, while protecting privacy through methods like selective disclosure and Zero-Knowledge Proof (ZKP). Selective disclosure allows sharing only specific attributes from credentials, while ZKP proves attributes without revealing their actual values [7], [8].

In a delegation use case, the delegates must be able to prove that the delegators have empowered them by issuing VM credentials. Therefore, the VP must at least include a VM and optional additional credentials as the verifier requires, and it must adhere to the relevant schemas and Standards. Additional credentials may be required to identify the delegate, or to prove certain aspects of its identity. Depending on the use case/workflow/process requirements, the VM verifier may require that the delegate has a specific certification or education, or is in a particular relationship with the delegator.

6) DIGITAL IDENTITY WALLETS, AGENTS, AND OFF-CHAIN DATA STORAGE

Digital identity wallets and agents are essential components of the SSI ecosystem, providing a means for participating in digital interactions, and enhancing digital identity systems' functionality, security, and UX. Usually, the two terms are used interchangeably [47]. However, they serve complementary but distinct roles, with different functionalities and purposes. **Wallets** are applications or services that provide secure storage and management of digital identities, including VCs, DIDs, and cryptographic material, giving the users control over their data [48]. They serve as User Interface (UI), and present a means for identification, authentication, and authorization, and are crucial in interaction with various services, resources, and entities [45]. On the other hand, **agents** are software programs or services that act on behalf of users to facilitate interactions within the SSI ecosystem. They automate processes such as VC issuance, presentation, and verification, manage interactions with other agents or services, and ensure compliance with user-defined policies and preferences [49]. **Off-chain data storage** can supplement local wallet storage, allowing the user data to be encrypted and stored in more conventional storage solutions (e.g., the cloud), enabling secure, scalable, flexible, and cost-effective user data management, while maintaining the benefits of decentralization and user control inherent in SSI.

To participate in the SSI ecosystem successfully, each entity must use an appropriate digital wallet. LEs should use enterprise wallets, while NPs should use digital identity wallets for identity holders. The wallets must support the basic SSI data flow, and provide functionalities related to the main SSI object (VCs, DIDs, and cryptographic keys) and their lifecycle [47], [50].

Regarding delegation, the wallet/agent[1] must integrate additional functionalities to support the delegation use cases. The delegators' wallet/agent must be able to obtain VM schemas, generate and issue VM credentials, send them to delegates, and, later, change their status if required. The delegates' wallet/agent must be able to review VM offers, receive and store VMs securely in the repository of their choice (control), request to perform delegated authority or access resources/services, and generate VP/VM proof responses based on the verifier's requirements. The verifiers' wallet/agent must be able to generate and manage VM schemas and policies, perform validation and verification checks, and connect to external service(s) or component(s), i.e., a Policy Engine, to obtain requirements for generating a VP/VM proof request and to obtain a delegation decision. Wallets/agents must be able to connect to external on-chain and off-chain data storage (VDRs), to obtain the information required for VM issuance, verification, and validation. Moreover, the VM verifier must display all the supported delegation use cases/workflows/processes on a website/portal from which delegators/delegates can choose from. The website/portal acts as a verifier's agent, and presents an entry point for the VM issuance and utilization. It automates data flow, and maps delegation use cases/workflows/processes with the associated schemas and policies.

[1] The term wallet/agent throughout the paper is used merely for simplicity.

7) VERIFIABLE DATA REGISTRIES

Verifiable Data Registries (VDRs) enable the storage and management of the information required for establishing trust [51] within a decentralized SSI ecosystem. They function as a trust anchor, a Single Source of Truth (SSOT), eliminating the dependences on a centralized system. They can be blockchain-based, or use other types of distributed ledgers or databases. VDRs are utilized commonly for storing and managing (i) public DIDs and their associated DID documents, (ii) information about public entities, (iii) schemas, (iv) status documents, and (v) proofs/evidence of data flows and interactions for auditing and timestamping purposes. Therefore, we can differentiate between a **DID Registry** serving as a Decentralized Public Key Infrastructure (DPKI) [16], [52], a **Public Institution Registry**, a trusted **Schema Registry**, a **Status Registry** (revocation, suspension, etc.), and a **Blockchain Anchor** [45]. Moreover, in delegation, a specialized type of VDR, **Policy Registry**, can be deployed to store and manage (vi) policies required for obtaining the delegation requirements and for the decision-making process.

Hence, to support delegation adequately, the use case/workflow/process owners, usually VM verifiers, should deploy all the mentioned VDRs in the initialization stages, to ensure the security and integrity of the VM credential issuance and verification process, and enable their users to have representatives or delegates. All the registries must be equipped with appropriate AC mechanisms, limiting entities to perform only authorized CRUD operations, increasing security, and preventing unauthorized access. Therefore, each involved entity must have the required authorization credentials to perform the required operations, and its wallets/agents component must be able to connect to the registries and obtain the required data.

8) GOVERNANCE FRAMEWORKS

Governance frameworks define the business, legal, and technical rules [53], policies, Standards, and best practices that govern the creation, issuance, management, transfer, validation, and verification of digital identities and VCs within an SSI ecosystem. They are crucial for the efficient functioning of the SSI ecosystem, as they ensure technical and non-technical trust and interoperability among the participants. They are fundamental for all the involved entities, and, thus, should be placed in the center of the trust triangle [47].

Within the scope of delegation, the **policies** are of key importance in the multiple phases of delegation. They provide the framework and rules for managing and controlling delegated authority or access securely and effectively. They are derived from (existing) use case/workflow/process rules, and can be business, legal, or technical in nature. They ensure that the delegated actions are carried out within the defined boundaries, maintaining security, privacy, trust, compliance, and operational efficiency. For delegation purposes, policies must be defined and registered by use case/workflow/process owners that enable delegation, usually by VM verifiers or entities that also define the schemas and legal rules for individual domains. Policies must be registered in the **Policy Registry** prior to VM utilization and verification. Moreover, the same entity that defines the policies should also deploy a service or a component, i.e., a **Policy Engine**, for automatic delegation decision-making that enables the evaluation of user claims against contextual data and policies. By deploying a Policy Engine and automating the enforcement of these policies within the SSI infrastructure, digital agents and systems can facilitate robust and reliable delegation processes, benefiting all the involved entities.

B. DELEGATION DATA FLOW

The subsequent section outlines the connection between the presented building blocks throughout a generalized SSI process flow,[2] which is adjusted to meet the delegation requirements. In addition to including the basic SSI building blocks described above, minor adjustments are required, as well as the inclusion of an additional component, i.e., the Policy Engine. Thus, this section presents and visualizes a simplified delegation process flow executed in the SSI ecosystem, focusing on data exchange.

The delegation data flow is depicted in Fig. 1, where we can observe the involvement of three main entities, namely, the delegator (VM issuer), the delegate (VM holder), and the VM verifier. Each entity must create at least one identifier before participating in the SSI ecosystem, by using a digital wallet/agent *(the green, blue and light purple rectangles)*, which, in addition to the VDR *(the longer, light brown rectangle)*, represents an essential component in the reference SSI architecture. In addition to the above, we can also see the inclusion of the Policy Engine *(the purple rectangle)* and the service or resource *(the purple-pink rectangle)* that the delegate wants to access, or perform some actions on behalf of the delegator.[3]

The establishment and management of the infrastructure, including VDRs (optionally), Policy Engine, and services/resources, is in the domain of the VM verifier, which is also responsible for the delegation process management. To support the delegation, the VM verifier must first (0) define a schema and policy for an individual use case/workflow/process, and register them in the Schema and Policy Registry. After this initialization phase, the delegator (1) can obtain the required VM schema from the Schema Registry, (2) issue a VM credential based on the schema, and transfer the VM to the delegate, who stores it in a digital wallet or external storage under his/her control. When the delegate wants or needs to act on behalf of the delegator,

[2]One of our previous studies [4] has already presented the generalized SSI process flow. Thus, we are building on existing work and extending it to support delegation.

[3]To ensure consistency and improve clarity, the images throughout the paper use the same colors for representing entities (and their wallets/agents), VDRs and Policy Engine.

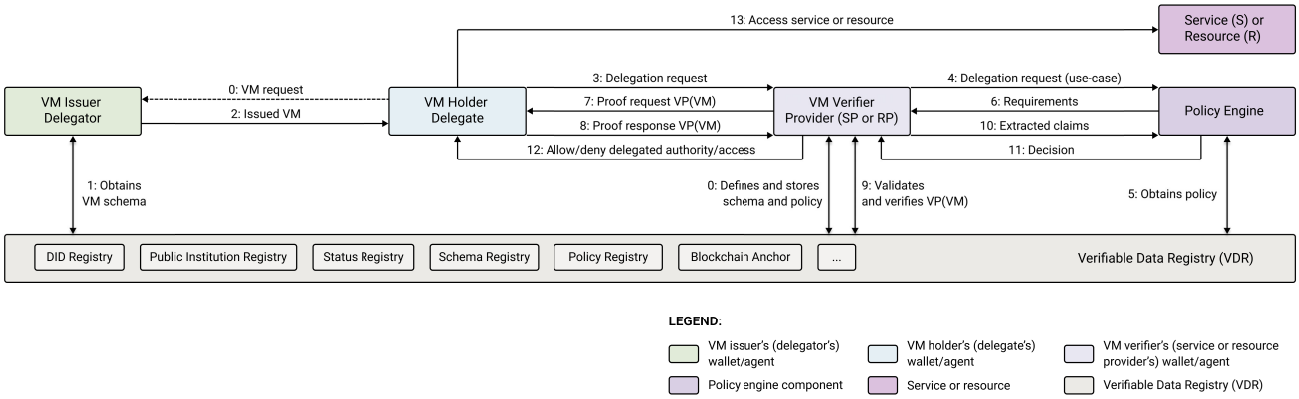


FIGURE 1. Simplified delegation data flow.

he/she (3) sends a delegation request to the VM verifier, who (4) forwards it to the Policy Engine for further processing. The Policy Engine (5) obtains the required policies from the Policy Registry (6) and returns the delegation use case requirements, based on which the VM verifier can (7) generate a VP/VM proof request, and forward it to the delegate. The delegate (8) responds with a VP/VM proof response, i.e., generates a VP that includes the required credentials, including a VM, proving that the delegator authorized him/her. After receiving the response, the VM verifier (9) processes, validates, and verifies the VP(VM), extracts the user data/claims, and (10) forwards them to the Policy Engine for further evaluation and decision-making. Therefore, the engine evaluates the user data/claims against contextual environmental data and the policy from the Policy Registry, and informs the VM verifier (11) on the delegation decision. Thus, the VM verifier can (12) approve or reject the delegated authority or access. Upon a positive outcome, the delegate can (13) access the service or resource on behalf of the delegate and perform the authorized actions.

C. DELEGATION PROCESS

The described delegation process can be divided into multiple phases, including several (sub)processes and process steps. To summarize, it requires the establishment, management, and maintenance of the process itself, the definition, issuance, and use of VM credentials by which the delegates demonstrate that they have the appropriate powers of the delegators, and the validation and verification of VM credentials, which enables informed decision-making. Moreover, the mechanisms for permanent or temporary delegation revocation enhance security and control. Therefore, we can differentiate between the following phases: (1) Delegation setup and maintenance, (2) Delegation credential definition and transfer, (3) Delegation credential utilization, (4) Delegation validation and verification, and (5) Delegation credential status change.

The different phases require the participation of different entities and/or components, and follow each other in a sequence, as illustrated in Fig. 2. For successful delegation, all phases *depicted with gray rectangles with solid borders* must be completed, while modifying the credential status is optional, and is *indicated with rectangles with dashed borders*. The Figure focuses solely on the phases of the process, omitting specific details that will be elaborated upon in the following sections. As a result, Figs. 3–8 provide detailed insights into each individual phase, as discussed in their respective sections, while the specifics of other phases remain excluded. For a comprehensive overview of the entire delegation process, including all stages and their details, please refer to the Appendix.

Each phase and included (sub)process has its objectives, and is critical to delegation. Each has requirements and prerequisites that must be met before execution, and each involves different entities with unique responsibilities that affect the integration and use of delegation-specific building blocks and components.

As part of this research, all the phases, processes, and sub-processes were analyzed, defined in detail, and modeled with sequence diagrams and BPMN models. However, individual phases (accompanied by sequence diagrams) are presented in the following, while the detailed (sub)processes (and BPMN models) are omitted due to the extensiveness of documentation. As for each phase and each (sub)process, we defined the objectives, involved entities, process steps, inputs and outputs, required building blocks, components, prerequisites, and requirements.

1) DELEGATION SETUP AND MAINTENANCE

Firstly, supporting and establishing a structured delegation process within the company or organization is essential. Delegation of authority or access, along with mandates, lacks significance if not endorsed and effectively integrated within the company, organization, service/resource provider, or, in general, an entity that enables delegation for different use cases, workflows, or (business) processes, acting as a VM verifier within the SSI ecosystem. Therefore, the delegation setup and maintenance phase are crucial for the entire delegation process. They serve as the foundation

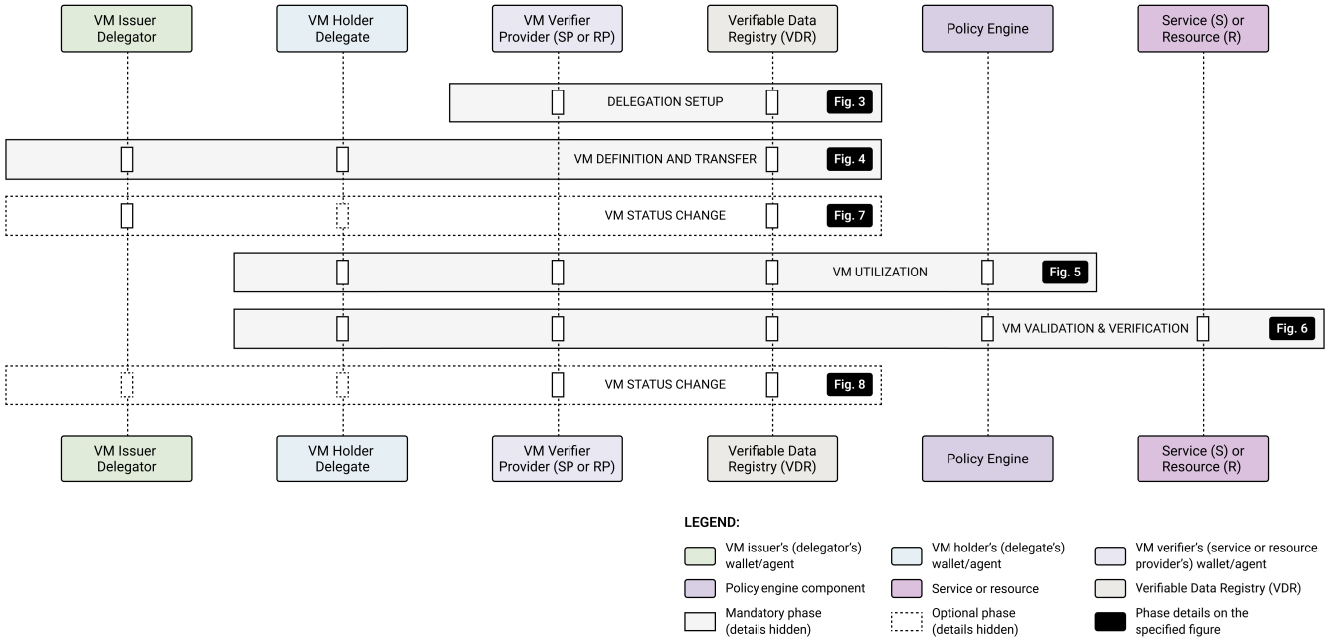


FIGURE 2. Delegation process overview.

of all the subsequent phases, and determine which use case/workflow/process will support delegation, and in what way. Thus, the main objective of this phase is to outline the delegation framework and extend the existing SSI ecosystem by (i) defining the delegation terms and conditions arising from the use case/workflow/process requirements, encoding them into policies, and storing them in Policy Registry, (ii) implementing and deploying a Policy Engine for verification, validation and decision-making purposes, and (iii) creating and storing VM schemas in the Schema Registry, used later for the VM credential generation, validation, and verification (Fig. 3). Defining schemas and policies and deploying a Policy Engine is crucial, as it is the basis for determining whether to authorize or reject the delegation of authority or access.

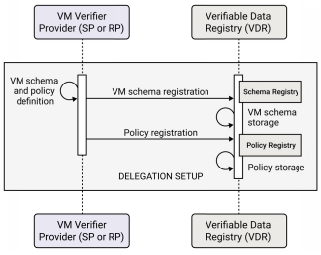


FIGURE 3. Delegation setup (Phase 1).

After supporting and integrating delegation within the selected use cases/workflows/processes, it is crucial that the VM verifier maintains the process, and updates schemas and policies when required. Therefore, upon changed circumstances, requirements, etc., the VM verifiers must define new schemas and policies (referencing older versions) and store them in appropriate registries, thereby ensuring compliance with their requirements, availability, and backward compatibility.

2) DELEGATION CREDENTIAL DEFINITION AND TRANSFER

We can begin defining, issuing, and transferring the VM credential only after the delegation setup phase has been completed successfully.

The main objective of this phase is to define, generate, and issue a VM credential based on the VM verifier's predefined VM schema stored in the Schema Registry. The VM credential is generated and issued by the VM issuer or delegator, and serves as evidence or proof that the VM holder or delegate can act on behalf of the delegator.

The VM credential determines the scope of the delegation, presents the information layer in the delegation model, and serves as the second level of restriction.[4] (i) It contains user data and identifies the involved entities, namely, the delegator, the delegate, intermediaries, or other involved entities (if present), resulting in a definition of the delegation relationship, connecting the VM credential and mentioned entities (binding the VM with the entities' DIDs). If the delegation involves using or accessing a certain thing (e.g., a resource/service), the credential must also identify it uniquely. Therefore, the VM credential must include a thing

[4]We can differentiate between two levels of delegate restriction, restrictions defined by (1) the VM verifier, expressed through policies and VM schemas (i.e., static restrictions reflecting the verifier's and use case/workflow/process requirements), and (2) the delegator, expressed in VM credentials (i.e., dynamic restrictions reflecting the delegator's requirements, needs and wishes).

identifier in addition to the delegator and delegate identifiers. For example, if the delegator (car owner) delegates the car's management and use (unlocking, driving, refueling, etc.) to the delegate, the VM must also contain the car's identifier (car DID). It specifies (ii) the permissions/grants/actions that empower the delegate to act on behalf of the delegate (delegates' empowerments, e.g., unlocking and driving), (iii) the restrictions/constraints, obligations, and other conditions (e.g., precise date-time value and location) that limit the delegated authority or access, (further) prevent the excess of privileges, reduce the possibility of abuse/misuse of privileges or power, and increase the control of the delegator. (iv) moreover, if required, it may also contain additional credentials, e.g., for the delegator identification (VID), or for proving other aspects of the delegator's identity (VAs). It is also important that the VM includes VC(s), proving that the delegator meets the required criteria, and has the underlying right to perform the action that he/she wants to delegate further.

Before issuing a VM credential, it is crucial that the delegator and delegate establish a two-way peer-to-peer connection and exchange DIDs. This is followed by the initiation of the VM credential issuance, which requires the delegator to obtain the appropriate VM schema from the Schema Registry, based on which it generates, signs, and issues the VM credential. After generating and issuing the VM credential, the delegator transfers the VM to the delegate, who stores the VM securely in the identity wallet or other repository under its control, and can use it later when it wants/needs to act or access a resource/service on behalf of the delegator (Fig. 4). The VM issuance process can be initiated either by the delegator, who may perceive the need to delegate authority or access, or by the delegate, who encounters limitations arising from existing access management mechanisms (varies on the use case).

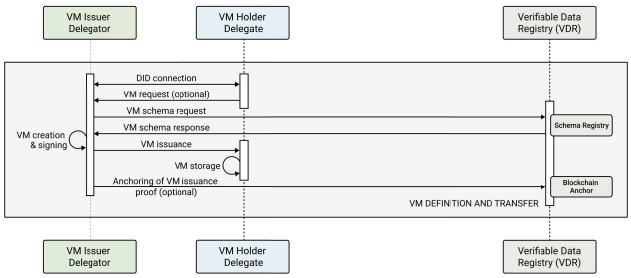


FIGURE 4. Delegation credential definition and transfer (Phase 2).

3) DELEGATION CREDENTIAL UTILIZATION

After issuing and transferring the VM credential in the previous phase successfully, the delegate can use the VM as proof of authorization or empowerment of the delegator (proof of authority or access delegation). The VM can be used immediately after acquisition, or later, as long as it is valid, or until suspended or revoked. Hence, the main objective of this phase is for the delegate to present the VP proof response containing the VM issued in the previous step together with additional credentials (if required) successfully to the VM verifier, whereby the VP is processed immediately and automatically in the next phase.

As shown in Fig. 5, this phase involves the cooperation of two entities: the VM verifier providing resources or services, and the delegate requesting access to a resource or service, acting on behalf of the delegator. The delegate and VM verifier must establish a peer-to-peer connection, and exchange DIDs prior to any other communication and data exchange. After receiving the request to perform the delegated authority or access, the VM verifier's agent redirects the request to the Policy Engine, that obtains the policy from the Policy Registry and determines the delegation requirements, serving as the basis for generating VP/VM proof requests. The latter specifies which credentials the delegate must include within the VP. In addition to the VM, which is crucial in the delegation process, supplementary credentials may be required, such as, e.g., VID, which serves to identify the delegate.

In summary, the critical step is for the delegator to create, sign, and issue a VP that includes a VM credential, and transfer it to the VM verifier for further processing and evaluation in the last mandatory phase of the delegation process.

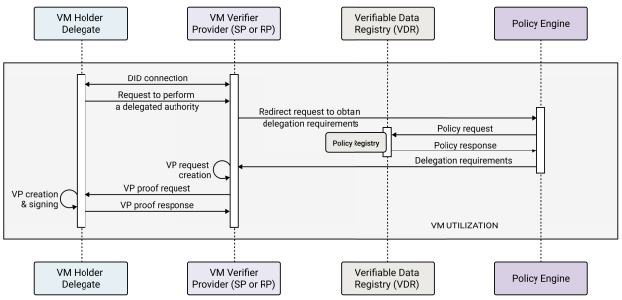


FIGURE 5. Delegation credential utilization (Phase 3).

4) DELEGATION VALIDATION AND VERIFICATION

The processing of the VP and included credentials, especially the VM credential, its validation and verification, and, last but not least, the decision-making, are vital for the delegation process, and require the completion of all the previously described phases. The main objective of this phase is for the VM verifier to determine if the delegate has the right credentials with the required data/attributes/claims to act on behalf of the delegator in the requested context, and enable or disable the delegated authority or access.

As delegates communicate directly with the VM verifiers, representing an intermediate link between the delegator and the verifier, they present the information layer within the SSI ecosystem, serving as a source of truth regarding themselves and the delegators. Therefore, they are responsible for providing all the necessary information needed as proof, to demon-

strate entitlement to the authority or access. They must have all the required credentials in their digital wallet to prove that they have been authorized/empowered appropriately (via the VM credential) and that the delegators meet the criteria for performing the authority or access they want to delegate further.

Therefore, this phase consists of two crucial steps: VP proof processing and decision-making. The first step represents the established process of verifying credentials within the SSI ecosystem. It refers to the validation and verification of the VP and included VM (as well as other credentials when present). The second step extends the basic SSI flow by including a component, the Policy Engine, responsible for evaluating the extracted user claims (from the VM credential issued in Phase 2) against contextual data and predefined policies (defined in Phase 1). Before extracting data from the VM and transferring it to the Policy Engine, it is necessary that the VM verifier's agent checks the VM's automatically (i) compliance with the VM schema, to ensure it contains the required data in the correct format, (ii) the validity period, to ensure it has not expired, (iii) its status, to ensure it has not been revoked or suspended by an authorized entity, (iv) its integrity and authenticity by verifying the VM's signatures, and, additionally, evaluate (v) the trust in the issuer's identity. Only then comes the data extraction and processing. Thus, the extracted user data are sent to the Policy Engine, which, in addition to the user data, obtains the environmental contextual data and policy from the Policy Registry. The engine evaluates the user data against the policy and contextual data, and decides whether to approve or deny the delegated authority or access. Optionally, it anchors a decision on the Blockchain Anchor for timestamping and auditing purposes. Depending on the implementation, it can provide simple true/false or allowed/denied answers, or it can generate a VAuthz, (access) token, or any other form of authorization/access credential that is later forwarded to the delegate, who can utilize it for performing delegated authority or access (Fig. 6).

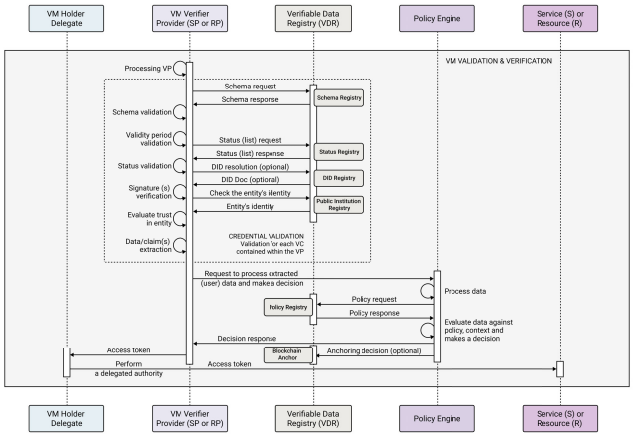


FIGURE 6. Delegation validation and verification (Phase 4).

5) DELEGATION CREDENTIAL STATUS CHANGE

Last but not least, the objective of the VM status change process (revocation, suspension, etc.) is to enable entities (mainly delegators, and, in some cases, VM verifiers) (i) to have control over the issued VM credentials, and (ii) to be able to revoke their validity in the event of a change in data, circumstances, a detected error, or for any other reason, (iii) to limit the activity of delegates, and (iv) to prevent possible abuse and unwanted behavior.

This phase is optional, and can be performed by the delegator (Fig. 7) at any time after VM issuance and transfer (Phase 2). On the other hand, a change of status by the VM verifier (Fig. 8) may occur after VM validation and verification (Phase 4) if specified within the VM credential by the delegator, or if abuse or other unwanted behavior is detected. However, both entities must be authorized appropriately to change the VM status.

Regardless of the entity performing the status change, this phase includes (i) selection of the VM, extraction of the status document and Status Registry ID, (ii) obtaining and defining/changing the status (and additional information if required) in the status document, and storing it in the registry, (iii) optional storage of proof of change for timestamping and auditing purposes, and (iv) optional notification of the delegator and delegate, as displayed in Figs. 7 and 8.

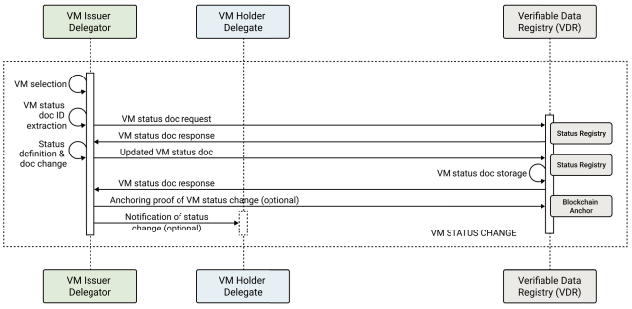


FIGURE 7. Delegation credential status change triggered by the delegator (Phase 5).

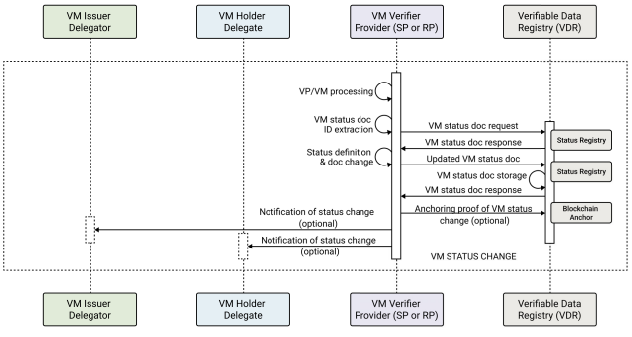


FIGURE 8. Delegation credential status change triggered by the VM verifier (Phase 5).

D. DELEGATION-SPECIFIC COMPONENTS

Delegation requires the integration of the following: A VM Schema Generator, a VM Credential Generator, a VM Credential Manager, a VM Credential Validator, a Verification Engine, a Policy Generator, and a Policy Engine, whereby some include several subcomponents. Each is responsible for performing specific operations, and has inputs and outputs that result in the final outcome, as shown in Table 4. Moreover, each operates with different SSI data objects, interacts with external data repositories,[5] and is used in different delegation phases and processes.

In the upcoming sections, each component will be described and graphically presented. Whereby optional elements will be *italicized or/and marked with a dashed line*, a *parallelogram* will indicate data objects, and a *cylinder* will indicate external data repositories (Figs. 9 - 15).

1) VM SCHEMA GENERATOR

The VM Schema Generator must allow the VM verifiers to define, generate, and register/store VM schemas in the Schema Registry, where they are available for later use by the VM Credential Generator, the VM Credential Validator, and the Verification Engine. When schema updates are required, the verifiers must generate a new VM schema that references the older version (to ensure backward compatibility), and register/store it in the Schema Registry.

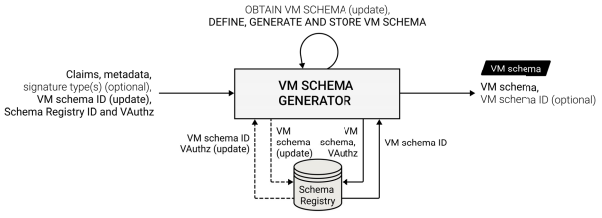


FIGURE 9. VM Schema Generator component for generating, updating, and storing VM schemas.

The essential component's inputs include (i) the required data/attributes/claims (the required properties with corresponding formats), metadata, and, optionally, signature type(s). The claims reflect the delegation process requirements or use case/workflow/process details, considering the specifications and existing policies. When updating the VM schema, the input is also the VM schema identifier, allowing the VM verifier to retrieve and update the current version, (ii) the Schema Registry unique identifier (location), enabling the VM verifier to register the generated VM schema in the selected repository, and, later, obtain and update it when required. The main outputs of the component represent a defined VM schema and, optionally, a unique VM schema identifier (Fig. 9).

[5]External data repositories (VDRs) must be protected appropriately with one of the AC mechanisms. Therefore, when the component must connect to and perform certain CRUD operations (e.g., storing SSI data object, etc.), its input must include a data object(s) (e.g., VM schema ID) and registry identifier (e.g., Schema Registry ID) and (optionally) an appropriate authorization credential (e.g., VAuthz) provided by the executing entity.

2) VM CREDENTIAL GENERATOR

The VM Credential Generator must allow the VM issuer (delegator) to create/generate, sign, and issue a VM credential that conforms to the W3C Standards (W3C VC data model) and the appropriate VM schema obtained from the Schema Registry. The issued VM is then transferred to the VM holder's (delegate's) digital wallet/agent, where it is validated and verified by the VM Credential Validator. Moreover, if required, it should allow the VM issuer to anchor proof of VM issuance to the Blockchain Anchor for timestamping and auditing. When VM credential updates are required, the VM issuers must revoke (VM Credential Manager) the existing VM credentials and generate new ones.

The component's direct inputs include (i) the VM schema identifier, to obtain the required VM schema that presents indirect input, outlining the structure and content of the VM credential, (ii) data/attributes/claims describing the VM subject, data limiting delegation scope, specifying **roles/attributes, permissions/grants, constraints/restrictions/limitations, and other conditions and obligations of delegation** (usually within a `credentialSubject` property). If required, the VM can also include additional credentials (such as VIDs, VAs), which the verifier requires (as defined in the VM schema), (iii) entities' identifiers and identity information, that enables the VM verifier to identify the delegator and delegate uniquely (and other intermediaries if required), (iv) metadata (optionally). However, the metadata are usually generated automatically, based on the VM schema and contextual data, (v) the VM issuer's (delegator's) cryptographic material (private key), for signing the VM credential to ensure authenticity, integrity, and tamper resistance. The main outputs of the component represent a digitally signed and issued VM credential, representing the delegation rights and constraints, binding the delegator and delegate to the VM credential, including information related to the issuance and status of the credential, etc., and, optionally, a unique VM credential identifier (Fig. 10).

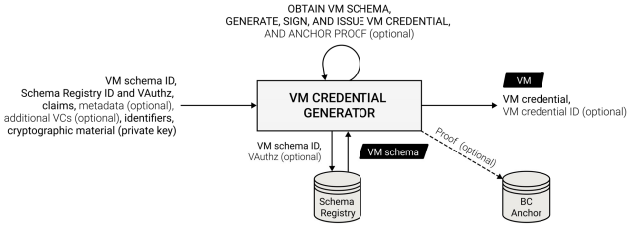


FIGURE 10. VM Credential Generator component for generating, signing, and issuing VM credentials.

3) VM CREDENTIAL MANAGER

Changing the status of a VM credential is crucial in the delegation process, due to various reasons such as changed

circumstances, requirements, (user) data, detected abuse, misuse or error, etc. Therefore, a component that enables revocation, suspension, or other type of status change is essential. Therefore, VM credentials must include a reference to the Status Registry, and a status document to enable an authorized entity to check and/or update the VM's status (document) if or when required. Hence, the VM Credential Manager must allow an entity, the VM issuer (delegator), and, in some cases, the VM verifier, to change the status (revoke, suspend, etc.) of the issued VM credential easily, and, optionally, notify the VM holder (delegate) about the status change. Thus, VM status in the Status Registry can be changed accordingly. Moreover, the VM's validity can be verified appropriately, and the VM credential can be accepted or rejected (by the VM Credential Validator and Verification Engine) if its status in the Status Registry is revoked or suspended by the authorized entity. If required, the proof of VM status change can also be anchored on the Blockchain Anchor.

The component's inputs are (i) VM (status document) identifier for updating the VM' status in the Status Registry, (ii) information about the status of the VM (suspended/revoked/etc.), (iii) entity's DID, and (iv) optionally, additional information regarding status change, such as a reason for status change, circumstances, etc. The main outputs of the component are the VM's unique identifier, together with its current/changed status or success/error message (Fig. 11).

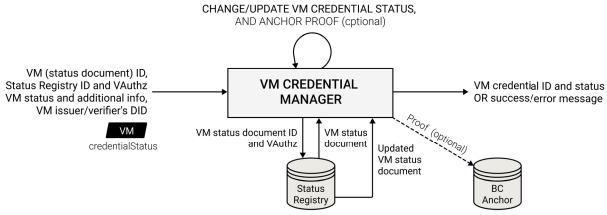


FIGURE 11. VM Credential Manager component for updating/changing the status of VM credentials.

4) VM CREDENTIAL VALIDATOR

The VM Credential Validator must allow an entity, the VM holder (delegate), and the VM verifier to validate and verify the received VM credential automatically, to ensure its authenticity and integrity before its acceptance and storage (the delegate's perspective), or before data extraction and transfer to the Policy Engine for further evaluation and decision-making (the verifier's perspective). Moreover, if required, the component should enable anchoring proof to the Blockchain Anchor for timestamping and auditing.

The component's direct input is a VM credential referencing data object(s) from external VDR registries, which is required for validation and verification. Indirect inputs are, thus, obtained from the VM credential, and include a VM identifier, a VM schema identifier for obtaining a VM schema, a VM status document identifier for obtaining a VM status document, DIDs for DID resolution and obtaining DID document(s), specifying public keys, endpoints, etc., and for identifying LEs, and establishing trust, proof values for signatures, and integrity checks, and a validity period for checking the VM's validity. The main output of the component is a success or failure message regarding the validity of the VM credential (e.g., true or false, valid or not valid).

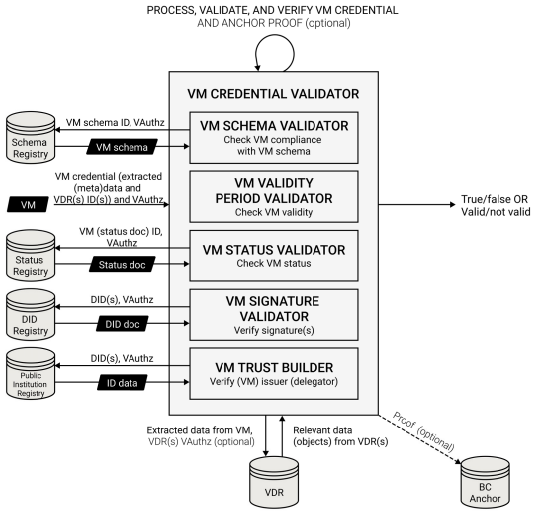


FIGURE 12. VM Credential Validator component with subcomponents for validating and verifying VM credentials.

The above inputs suggest that the component may include several subcomponents responsible for validating different parts of a VM credential. Each subcomponent is responsible for performing a different operation, has different inputs and outputs, and connects to various external registers to obtain the relevant data required for validation and verification checks. The VM Credential Validator should include (Fig. 12):

1) **A VM Schema Validator**, obtains the relevant VM schema from the Schema Registry, referenced in the VM credential (`credentialSchema` property). It checks the VM's compliance with the VM schema, to ensure the VM is structured correctly and contains all the required data in the required format, enhancing interoperability and trust.
2) **A VM Validity Period Validator**, can determine if the VM is valid or expired (appropriate range of the validity) based on the issuance and expiration date-time value (`validFrom` and `validUntil` properties) in the VM credential.
3) **A VM Status Validator**, retrieves the VM credential status information, e.g., from a status document referenced in the VM credential (`credentialStatus` property) and maintained in the Status Registry, to ensure that the VM is valid and has not been revoked or suspended by the authorized entity.

4) **A VM Signature Validator**, which is responsible for verifying the cryptographic signature(s), ensuring the VM's authenticity and integrity. It resolves the issuer's DID (from the VM) to the DID document, to retrieve the relevant data required for signature(s)' verification, and VM integrity checks to ensure that the content of the VM has not been altered since its issuance.
5) **A VM Trust Builder**, which is responsible for verifying the identity and authority of the VM issuer (delegator) by obtaining and verifying data either from (i) the Public Institution Registry (trusted issuers, LEs), or (ii) the additional credentials (VID, VA) included in the VM credential (personal issuer, NPs), ensuring that the VM comes from the trusted source.

5) VERIFICATION ENGINE

The Verification Engine must allow an entity, the VM verifier, to (1) validate and verify the received Verifiable Presentation (VP) and included VCs (including VM credential issued by the delegator) automatically without the verifier's intervention, to ensure its authenticity and integrity, and (2) extracts metadata and user data when all the verification checks are passed successfully. Thus, it performs VP validation and verification before the credential(s) and data extraction and data transfer to the Policy Engine for further processing, evaluation, and decision-making. A VP can include multiple credentials, but the key to the delegation process is verifying and extracting data from the VM credential(s), which serve as the delegation proof.

Therefore, the Verification Engine must include the VM Credential Validator component with all the subcomponents described in the previous chapter, which enables verification of the VM credential's compliance with the VM schema, verification of the VM's validity, status, signatures, credibility, and the trustworthiness of the VM issuers.

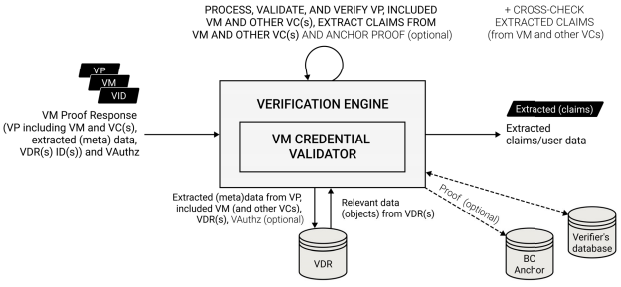


FIGURE 13. Verification Engine component with the VM Credential Validator subcomponents for validating and verifying Verifiable Presentations and included VM credential(s).

The component's direct inputs are a VP/VM proof response containing various credentials (VAs, VIDs, VMs, depending on the VM verifier requirements) with metadata and user data/attributes/claims. Indirect inputs are, thus, obtained from each credential/presentation, and include a credential identifier, a schema identifier for obtaining the schema, a status document for obtaining the status document, DIDs for DID resolution, and obtaining the DID documents, specifying the public keys, endpoints, etc., and for identifying the LEs and establishing trust, proof values for signatures and integrity checks and the validity period for checking the validity. The main outputs of the component are the extracted user data/attributes/claims, that are forwarded to the Policy Engine for further evaluation and the decision-making process. However, the extracted data can be cross-checked additionally by consulting the verifier's off-chain data stores to enhance trust (Fig. 13).

6) POLICY GENERATOR

The Policy Generator must allow the VM verifiers to define, generate, and register/store policies in the Policy Registry, where they are available for later use by the Policy Engine for decision-making and VP/VM proof request generation. When policy updates are required, the verifiers must generate a new policy referencing the older version, to ensure backward compatibility, and register/store it in the Policy Registry.

The component's direct inputs include (i) possible **roles/attributes, permissions/grants, conditions, obligations, constraints/restrictions/limitations**, etc., and (ii) **business rules**, both reflecting the delegation use case/workflow/ process requirements. When updating the policy, the input is also the policy identifier, allowing the VM verifier to retrieve and update the current version, (iii) the Policy Registry unique identifier (location), enabling the VM verifier to store/register the generated policy in the selected repository and later obtain and update it when required. Indirect inputs include (iii) delegation use case/workflow/process detailed description and requirements, that are encoded in policies with the selected policy language. The main outputs of the component represent a defined policy in the chosen policy language, and, optionally, a unique policy identifier (Fig. 14).

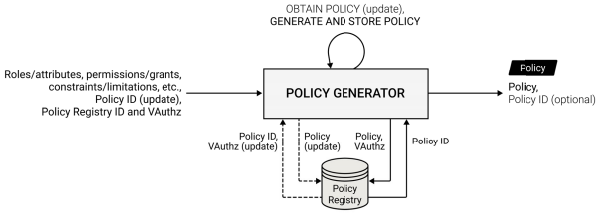


FIGURE 14. Policy Generator component for generating, updating, and storing policies.

7) POLICY ENGINE

The differentiation between the Verification Engine and Policy Engine allows the separation of concerns. The former is thus responsible for verifying credentials and extracting data, while the latter is responsible for making and enforcing decisions. The Policy Engine is essential in the delegation process, enabling the VM verifiers to (i) define and communicate delegation requirements, and (ii)

make and enforce verified, reliable, and trusted decisions to allow or deny delegated actions. The Policy Engine presents extensions to the basic SSI model, enhancing control and flexibility to (business) processes and workflows, and addressing marginal use cases, such as IIC.

(i) Firstly, the Policy Engine must obtain a policy for a particular delegation use case/workflow/process, and define the delegation requirements based on this policy, enabling the VM verifiers to generate VP/VM proof requests, encapsulating the information needed for decision-making. Therefore, the component's direct input is a delegated authority or access request, identifying the delegation use case/workflow/process and policy identifier. The main output of the component is the delegation requirements or VP/VM proof request details regarding the required data/attributes/claims and/or credentials that need to be included in the VP, together with the data/credential types. (ii) secondly, the Policy Engine must allow the VM verifiers to obtain, process, and evaluate the user data/attributes/claims (from the VM) against environmental contextual data and predefined policy (use case/workflow/process rules) obtained from the Policy Registry, to make a decision regarding the delegated authority or access. It must enable the verifier to determine if a delegate can act on behalf of the delegator. Moreover, if required, it should allow the verifier to anchor the decision (proof) to the Blockchain Anchor for timestamping and auditing purposes. The component's direct inputs include (a) the user data/attributes/claims extracted from the VP/VM, (b) the contextual data obtained from the environment, and (c) the policy identifier. The main outputs of the component represent a decision regarding the delegated authority or access. Depending on the use case/workflow/process requirements, this decision can be simple or complex, resulting in a true/false or allow/deny answer, or in the generation of an authorization credential (VAuthz), token, or any other form of authorization transferred to the delegate (Fig. 15).

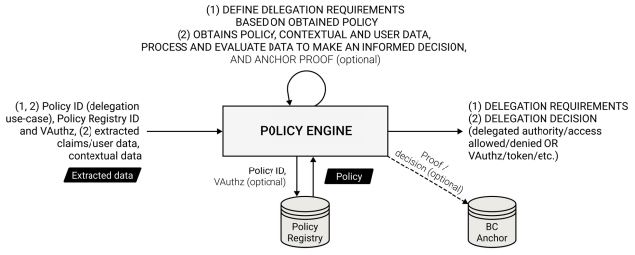


FIGURE 15. Policy Engine component for (1) defining the delegation requirements based on the predefined policy, and (2) evaluating the user and contextual data against the policy to decide regarding the delegated authority or access.

E. WALLETS/AGENTS

To enable the delegation process in the SSI ecosystem, it is critical that all the entities, i.e., delegator, delegate, and VM verifier, have a digital identity wallet/agent that supports the core SSI functionalities related to the acquisition, storage, management, and exchange of underlying SSI data objects, but also includes functionalities that support delegation and include the delegation-specific components and sub-components described above.

However, a wallet/agent for different entities requires the inclusion of different components, depending on the entities' roles in the SSI ecosystem. Therefore, (1) the VM issuer (delegator) needs a VM Credential Generator and a VM Credential Manager, (2) the VM holder (delegate) needs a VM Credential Validator, (3) the VM verifier needs a VM Schema Generator, a Policy Generator, a Policy Engine and a Verification Engine (including a VM Credential Validator), and a VM Credential Manager, as depicted in Fig. 16.

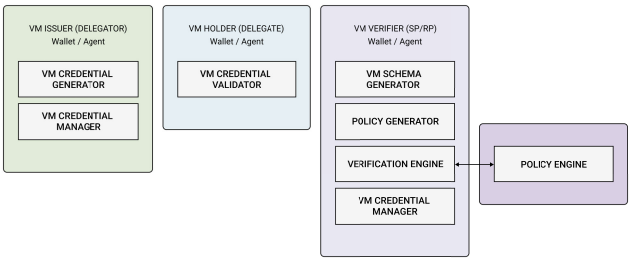


FIGURE 16. Delegation-specific components required by each entity (VM issuer, VM holder, and VM verifier).

IV. MODEL VALIDATION

After the design phase of the novel authority and access delegation model, we proceeded with its validation. The validation is based on a case study, precisely, on the use case, **enrollment into first-cycle undergraduate study programs at a higher education institution** *(in the following enrollment and the Faculty)*. However, the model could also be validated across various use cases in private and public sectors, particularly in scenarios where service or resource providers enable users to designate representatives to act on their behalf. For instance, an individual may delegate a friend to pick up a book from the library, collect a package from the post office, retrieve medication from the pharmacy, make healthcare decisions in critical conditions, etc. Moreover, the model's applicability extends to numerous other use cases within different sectors, where service or resource providers allow users to designate representatives to act on their behalf.

Before validation, we analyzed the existing official documentation[6] and the entire enrollment process carefully, from the application to the official admission, focusing on the parts the novel delegation model could support. Afterward, we applied the model to the student enrollment process by simulating its utilization systematically through

[6]The application and enrollment procedure is formal, established, and well-documented. It is determined in the *Call for Enrolment into first-cycle undergraduate study programs* for each academic year. Additional information and documentation are publicly available online on the official websites of Slovenian higher education institutions, such as the Ministry of Higher Education, Science and Innovation, and the websites of universities, individual Faculties, etc. [54], [55], [56].

TABLE 4. Delegation-specific (sub)components with supported operations, associated inputs, outputs, and results.

Component	Operations	Inputs	Outputs (and results)	Registries
VM Schema Generator	Define, generate, *obtain and update*, store VM schema	Claims, *metadata*, and *signature type(s)*, Schema Registry ID and VAuthz, *VM schema ID (update)*	VM schema, *VM schema ID* Result: VM schema stored in the VM Schema Registry	Schema Registry
VM Credential Generator	Obtain VM schema, generate, sign, and issue VM, *anchor proof*	VM schema ID, Schema Registry ID and *VAuthz*, claims, *additional VCs*, *metadata*, identifiers, cryptographic material (private key)	VM credential, *VM ID* Result: Issued VM credential, *anchored VM issuance proof*	Schema Registry, *Blockchain Anchor*
VM Credential Manager	Change/update VM status, *anchor proof*	VM (status document) ID, Status Registry ID and VAuthz, VM status, entity's DID, additional information	VM credential ID, VM status OR success/error message Result: changed VM status, *anchored proof*	Status Registry, *Blockchain Anchor*
VM Credential Validator	Process, validate and verify VM, *anchor proof*	VM credential, extracted VM data and metadata, VDRs IDs and *associated VAuthz*	True/false or valid/not valid Result: Validated and verified VM credentials, *anchored proof*	Different VDRs, *Blockchain Anchor*
VM Schema Validator	Obtain VM schema, check VM compliance with VM schema	VM credential, VM Schema ID, *Schema Registry ID* and *VAuthz*	True/false or valid/not valid Result: Verified VM credential compliance with VM schema	Schema Registry
VM Validity Period Validator	Obtain and check VM validity period	VM credential, VM issuance and *expiration* date-time value (validFrom – validUntil)	True/false or valid/not valid Result: Verified VM credential validity period	/
VM Status Validator	Obtain and check VM status	VM credential, VM status document ID, Status Registry ID and *VAuthz*	True/false or valid/not valid or status with additional info (explanation, date-time, entity, etc.) Result: Verified VM status	Status Registry
VM Signature Validator	Obtain cryptographic material, verify the authenticity and integrity of the VM signature(s)	VM credential, proof(s), DID(s), DID Document(s) (public key(s), endpoint(s), etc.), DID Registry ID and *VAuthz*	True/false or valid/not valid Result: Verified VM signature(s)	DID Registry
VM Trust Builder	Obtain and verify the identity of the VM issuer	VM credential, DID(s), DID Document(s), *Public Institution Registry ID* and *VAuthz*, *additional VCs (VID, VA)*	True/false or (not)trusted Result: The identity of the VM issuer is verified	Public Institution Registry
Verification Engine	Process, validate, verify and extract claim(s) from VP, included VM and other VCs, *anchor proof*	VP, VCs (VM, VA, VID), VP/VCs extracted user data and metadata, VDRs IDs and *associated VAuthz*	Extracted user data/attributes/ claims Result: Validated and verified VP (and included VM and other VCs) and extracted user data	Different VDRs, *Blockchain Anchor*
Policy Generator	Define, generate, *obtain and update*, store policy	Roles/attributes, conditions, obligations, permissions/grants, constraints/constraints/restriction-s/limitations, Policy Registry ID and VAuthz, *policy ID (update)*	Policy, *Policy ID* Result: Policy stored in the Policy Registry	Policy Registry
Policy Engine	(1) Define delegation requirements (2) Obtain and evaluate data, make a decision, *anchor proof*	(1, 2) Policy ID, Policy Registry ID and VAuthz (policy), (2) extracted claims, contextual data	(1) Delegation requirements Result: Generated proof request based on requirements (2) Delegation decision (true/ false, allowed/denied, VAuthz) Result: Delegated authority or access is granted or denied	(1, 2) Policy Registry (2) *Blockchain Anchor*

an envisioned realistic scenario depicted in Fig. 17,[7] in which we can observe the high-level approach, crucial architectural components, and involved entities.

Although delegation could be applied both to the application and to the official student enrollment process at the Faculty, for illustrative purposes, we focused only on delegating the enrollment process, which, upon submission

[7]The Policy Engine is shown as part of the verifier wallet/agent for simplicity. However, for modularity purposes and the possibility of its utilization by multiple higher education institutions, it is preferably intended to be implemented as an external component, which is also considered in the following Figures.

of the required documents and mandate, is currently carried out physically at the Faculty, or, in some cases, electronically through the academic information system. Students can have delegates who enroll them into the Faculty. However, all enrollment steps must follow the official regulations accurately, the same as in the process carried out by the students themselves. In addition to the student's information, the delegate must also submit all the required documents necessary for enrollment and the student's written authorization, currently in the form of a completed and signed form, which must be accompanied by a copy of the student's identity document.

A. DELEGATED STUDENT ENROLLMENT

In the following section we present the application of the proposed delegation model to the use case of official student enrollment into the Faculty (in Slovenia), taking into account the current process and using fictional entities. We outline the use case by justifying the reason for the delegation, presenting the involved entities [E], assumptions [A], prerequisites [P], and enrollment delegation process through the phases from Section III-C.

As presented in Fig. 17, each entity, usually LEs, that wants to support delegation within a specific use case needs to define a VM schema and policy, which are critical to the execution of the process. In our case, the Faculty determines the VM schema and policy, as formal enrollment is currently regulated at the level of the individual Faculty or higher education institution. This means that the procedures, deadlines, and necessary documentation may vary. However, at a higher level, the VM schema and policy could also be defined by a parent institution, such as a university or ministry, and utilized by the Faculty for validation, verification, and decision-making. The same can be done by other institutions, such as libraries, banks, post offices, administrative units, municipalities, etc., who want to support the delegation for the various services and processes they provide to their users or customers, and, thus, act as verifiers in the presented flow. Afterward, Alice (the delegator) can delegate student enrollment into the Faculty to Bob (the delegate) by generating, signing, and issuing a VM credential based on the VM schema and passing it to Bob, who can present it to the Faculty upon official enrollment. The Faculty validates and verifies the VM credential and extracts the data, which are then evaluated using the Policy Engine.[8] The same approach can be utilized regardless of the delegation process an entity wants to support. The essential difference lies in the policy and VM schema that dictate the content and structure of the VM credential.

1) ENTITIES AND REASON FOR DELEGATION

E1: **Delegator or VM issuer:** student Alice (NP)
E2: **Delegate or VM holder:** Alice's brother, Bob (NP)
E3: **VM verifier:** Faculty (LE)

Reason for delegation: Alice cannot enroll in the desired first-cycle undergraduate study program at the desired Faculty, as she does not have access to the internet at the time of enrollment. So, she authorizes her brother Bob to proceed on her behalf.

2) ASSUMPTIONS [A] AND PREREQUISITES [P]

A1: **Faculty:** The application process has been completed successfully. Thus, Faculties can enroll students into the study programs officially. The enrollment procedure is well established and documented, and the basic process (without delegation) is already well supported within the SSI environment. Electronic enrollment for all admitted students is enabled by utilizing basic SSI building blocks. Thus, it can be extended to support delegation, allowing the students to have delegates if they cannot carry out certain actions themselves.
A2: **Alice:** Alice has been accepted to at least one study program as she meets the required criteria. She is well acquainted with the enrollment process, deadlines, conditions, and other requirements, including delegating enrollment to someone else. Thus, she obtained all the required credentials, and can continue with the (delegated) enrollment process.
A3: **Bob:** Alice informed Bob about the entire enrollment process, deadlines, conditions, and other requirements. Thus, he is well aware of his involvement, responsibilities, obligations, and consequences of not carrying out a delegated task. He agrees to represent and enroll his sister Alice in the desired study program at the Faculty.

1) **SSI ecosystem:** All the entities operate with the SSI ecosystem, utilizing the basic SSI building blocks described in Section III-A.
2) **Wallets/Agents:** All the entities must have a digital identity wallet through which they can participate in the SSI ecosystem, communicate, exchange data, identify and authenticate themselves, access services and resources, and delegate actions. Alice and Bob (NPs) must have a digital identity wallet for the holder, while the Faculty (LE) must have an enterprise wallet.
3) **Identifiers:** All the entities must have at least one DID, associated with the cryptographic keys. The Faculty (LE) must have public DID registered in the DID Registry (authentication) and the Public Institution Registry (identification). On the other hand, Ana and Bob, NPs, should use private, self-sufficient DID methods for their authentication.[9]

[8]Policy Engine can be integrated within the verifier's enterprise agent or implemented as an external component or service with which the verifier communicates.

[9]When issuing a VM credential by the, i.e., private issuer, in our case, Alice, we can use private DID methods such as did:key in combination with the VID. The VID should be chained in the issued VM credential for the VM issuer identification purposes. Since the VID is issued by a trusted issuer with its DID rooted in a publicly available registry, the VM verifier can establish trust in the VM issuer.

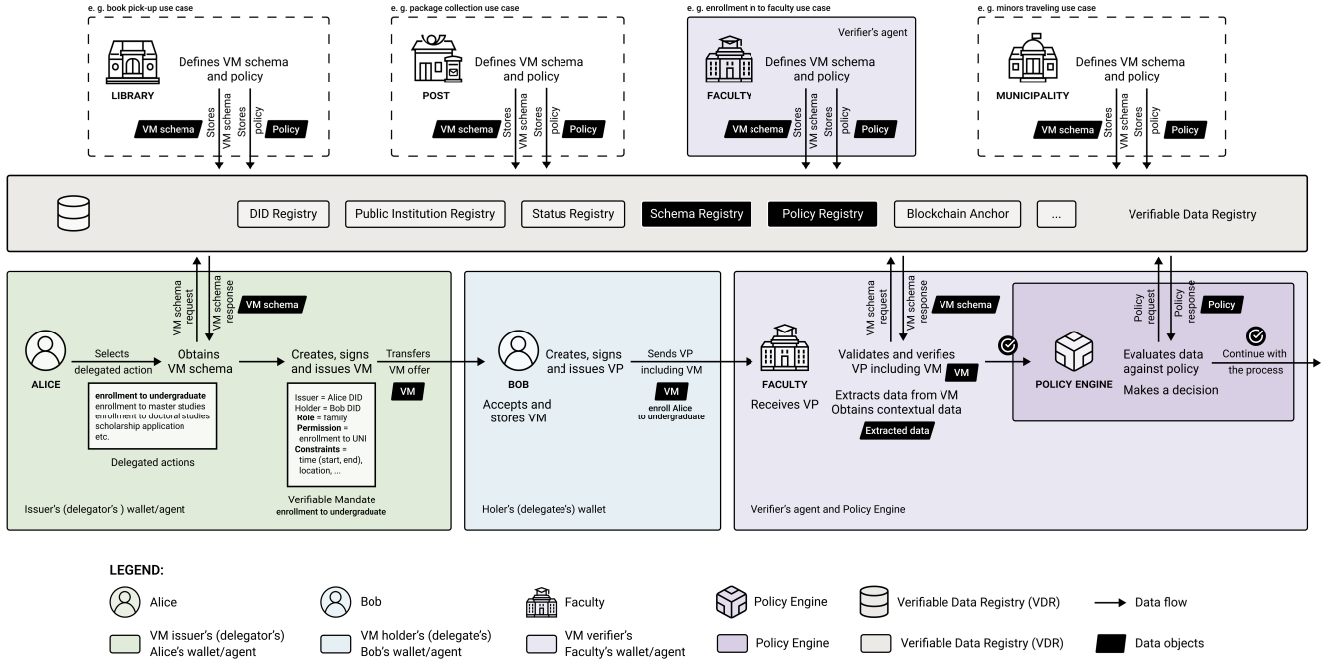


FIGURE 17. A high-level overview of delegating enrollment into first-cycle undergraduate study programs.

4) **Peer-to-peer connection:** Before issuing and transferring a VM, Alice, and Bob must exchange and resolve DIDs, and establish a mutual DID connection for secure communication (also an OpenID for Verifiable Credentials is possible), authentication, data and credentials exchange, encryption, and decryption. Consequently, when generating a VM, Alice can select Bob from the "established connections" list and designate him as her delegate. Thus, Alice's agent can successfully send a VM offer to Bob's agent as she previously obtained his DID, public key, endpoints, authentication methods, etc.
5) **Credentials:** Alice and Bob must obtain VID credentials (equivalent to a passport or ID card) from trusted issuers and store them in their digital identity wallets for identification purposes. Alice must include her VID in the VM, and Bob must present his VID together with Alice's VM inside the VP. Thus, the VM verifier can identify both despite them being NPs. In addition to the VID credential, before generating a VM, Alice must obtain (and store in her wallet) other VCs from trusted issuers, which are required for successful enrollment and are defined within the VM schema. The aforementioned allows her simply to select the VCs she wants to include when generating a VM credential.

3) PHASES AND KEY PROCESS STEPS

a: DELEGATION SETUP AND MAINTENANCE

When the Faculty wants to support the delegation of enrollment and enable students to empower delegates to carry out the enrollment process in their name, the Faculty must first establish the infrastructure, and define the VM schema and delegation policy to automate validation, verification, and decision-making (Fig. 18).

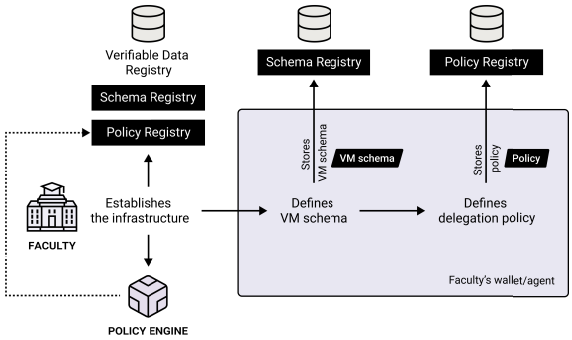


FIGURE 18. The Faculty establishes the required infrastructure and defines the VM schema and policy.

1) **Establishment of infrastructure:** The Faculty establishes the infrastructure, which includes deploying or connecting to (i) the required registries that are crucial for establishing trust, (ii) a component (or service) for delegation decision-making (Policy Engine), and (iii) an SSI agent or website/portal that presents the entry point for initiating the delegation of student enrollment including VM credential issuance and utilization, etc. It serves as a directory, listing all the provided delegation use cases (including delegated student enrollment) while mapping each to its associated VM schema and policy.
2) **VM schema and policy definition:** The Faculty defines the VM schema and policy, and stores them in the Schema and Policy Registry, respectively. The VM schema defined by the Faculty is critical in the creation (VM issuer,

Alice) and verification (VM verifier, Faculty) of the VM credential, as it determines the structure and content (including the format and data types) of the credential for delegated enrollment into the Faculty. On the other hand, the defined policy must reflect the business/process logic and requirements of the delegated enrollment process. It must contain rules allowing the Faculty to decide to approve or deny a VM credential. Upon a positive result, the Faculty can proceed with enrollment and enroll Alice into the desired program.

b: PH2: DELEGATION CREDENTIAL DEFINITION AND TRANSFER

When Alice wants to delegate enrollment into the Faculty to her brother Bob, she must first obtain the VM schema and then generate, issue, and transfer a VM credential to Bob, who must accept it and store it in his digital wallet (Fig. 19).

1) **VM generation, issuance, and transfer:** Alice visits the Faculty's website/portal (agent) and selects the "Enrollment delegation OR Mandate issuance" *[PH1.1]* (use case selection). Her agent obtains the VM schema from the Schema Registry *[PH1.2]*, and generates a form for issuing the VM credential displayed in her wallet. She enters all the required information to create a VM credential, including her personal and contact data, Bob's personal data, and his identifier (DID), designating him as her delegate and the recipient of the VM credential *[PR4]*. While the VM shema already defines delegated action (enrollment into the Faculty), Alice must specify the Faculty, study program, year of enrollment, electives, and any additional restrictions, such as the time and location of the VM utilization, thereby limiting the use of the credential contextually. She must also attach additional credentials (VC selection), namely, a VID for identification *[PR5]*, a photo for issuing a student card, a certificate of payment of the enrollment fee, a certificate of completed high school, and a certificate of graduation *[PR5]*. By clicking the "Issue Verifiable Mandate" button, followed by confirmation and authentication (optional), Alice's agent generates and signs digitally the VM credential utilizing her private key associated with her DID, issues and sends the VM offer automatically to Bob's agent via a secure, previously established DID connection *[PR4]*.
2) **VM reception and storage:** Bob receives a push notification that he has received a VM offer, which he can review and accept or reject. After acceptance, the VM is stored in his digital identity wallet (or external storage). Alice is notified of the successful acceptance, and her agent can record the issuance of the VM on the Blockchain Anchor (proof of issuance). When enrollment time arrives, Bob can use the issued VM to enroll Alice in the Faculty.

c: PH3: DELEGATION CREDENTIAL UTILIZATION

When Bob wants to use the VM credential to enroll his sister Alice into the Faculty, he must first visit the Faculty's website/portal, obtain the VP requirements, and generate, issue, and transfer the VP to the Faculty's agent (Fig. 20).

1) **Request to perform delegated enrollment:** On the day of enrollment, Bob can represent Alice and enroll her into the Faculty in the desired program, as she determined within the VM credential [PH2.1]. Bob visits the Faculty's website/portal (agent) and selects the "Enrolment with mandate OR Mandate utilization" *[PH1.1]* (use case selection), thereby initiating the delegated enrollment process, establishing a two-way DID connection with the Faculty's agent, and submitting a request to perform delegated enrollment utilizing the VM credential.
2) **VP/VM proof request generation:** The Faculty's agent forwards the request to the component for managing delegated actions (Policy Engine *[PH1.1]*), which retrieves the relevant policy from the Policy Registry *[PH1.2]*, based on which it determines the requirements for the delegated enrollment process, i.e., the VP requirements. The Faculty's agent can then create a VP/VM request based on the information obtained regarding which data and credentials need to be included in the VP, and forward it to Bob's agent.
3) **VP/VM proof response generation:** Based on the received request, Bob's agent generates a VP/VM response, which includes the VM issued by Alice, proving that Alice authorized Bob to perform delegated enrollment *[PH2.1]*, and Bob's VID, proving his identity and serving as proof of legal age *[PR5]* (VC selection). After reviewing the presentation, Bob initiates VP generation, digital signing, issuance, and VP transmission to the Faculty's agent by clicking the "Send" button (similar to

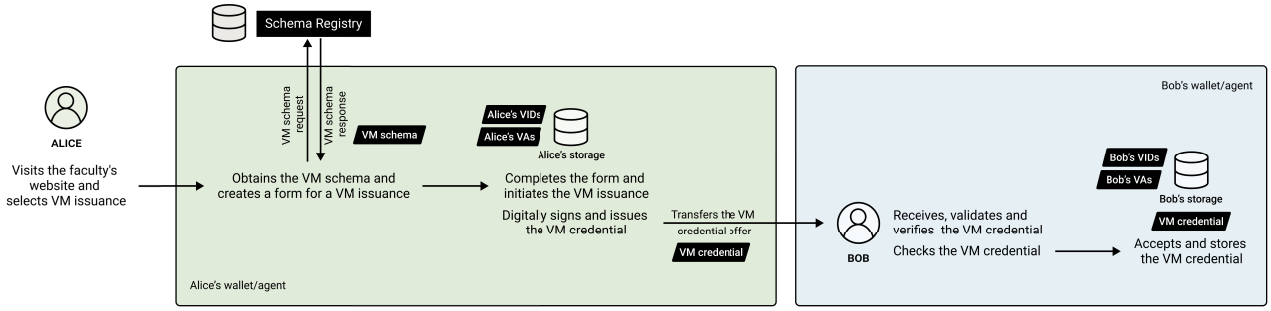


FIGURE 19. Alice creates, signs, and issues the VM credential, and she transfers it to Bob.

VM issuance *[PH2.1]*). This is followed by validation and verification of VP(VM) and data extraction *[PR4]*.

d: PH4: DELEGATION VALIDATION AND VERIFICATION

After receiving the VP/VM response, the Faculty's agent starts processing the received presentation and included credentials (VM and VID). It validates, verifies, and extracts data from the credentials and forwards them to the Policy Engine, which evaluates the extracted data against the policy and contextual data, and decides whether Bob has the appropriate authorizations to enroll Alice into the Faculty in the desired program (Fig. 21).

1) **VP(VM) validation and verification:** The Faculty's agent will first validate and verify the VP, its compliance with the schema, and Bob's digital signature. Then, it will extract, validate, and verify all the included credentials (Bob's VID and VM issued by Alice, as well as all the credentials included within the VM) separately. For each, it will check compliance with the schema, the validity period, status, digital signature(s), and issuer's identity. Digital signature verification depends on the DID method utilized, and the issuer verification depends on whether it is a public-trusted (VIDs and VAs issued by LE) or private-personal issuer (VM issued by NP - Alice). In the case of Bob's VID issued by a trusted issuer (LE), the agent checks the Public Institution Registry, where the data about public entities are stored. In the case of a VM issued by Alice (NP), whose data is not recorded in a publicly accessible registry (privacy, GDPR), verification of Alice's VID (included in the VM) issued by a trusted issuer (LE) is required, as proof of her identity.
2) **Data extraction:** After successful validation, the Faculty's agent extracts the data necessary to confirm or deny delegated enrollment from the credentials. The agent extracts (i) the personal data of Alice and Bob (from the VM and VID), to check if the data from the VM and VID coincide, (ii) permission regarding delegated action (enrollment into the Faculty), (iii) restrictions for the utilization of the VM, namely: the Faculty, study program, academic year, the time and location of the VM utilization, and (iv) the type of credentials and data received.[10] The extracted data are passed along with the contextual data (such as the date and location of the request, etc.) to the Policy Engine *[PH1.1]*.
3) **Data evaluation and decision-making:** Upon obtaining a request for data processing and decision-making regarding delegated enrollment together with the data, the Policy Engine obtains the relevant policy from the Policy Registry *[PH1.1]*. This is followed by evaluating the user and contextual data according to the obtained policy. The Policy Engine determines (i) the adequacy of the identification data, since the data from the VID and VM coincide, (ii) that the request to perform delegated enrollment was initiated within the powers/permissions, and, (iii) the constraints/restrictions/limitations defined in the VM, whereby the extracted data conform to the contextual data and defined policy, and (iv) it discovers that all the required data and additional credentials for enrollment have been provided. Therefore, it determines that Bob is duly empowered to enroll Alice into the Faculty in the desired program in the current academic year (VM). The decision is then forwarded to the Faculty's agent, who notifies Bob about the positive outcome, and, optionally, records the proof of the decision on the Blockchain Anchor. The agent can continue with the enrollment process.

[10] We are not interested in the content of the credentials included within the VM at this point, as we are trying to determine if Bob can enroll Alice and not actually enroll her yet. Consequently, all that is needed is information that, in addition to the authorization, the relevant data and credentials are provided, which will enable the Faculty to complete the enrollment.

4) VERIFIABLE MANDATE

In the following subsection, we present an example of a Verifiable Mandate (VM) credential for delegated student enrollment to the Faculty in JSON format, from which its structure and content can be observed (Listing 1). The VM contains fictional data, but is consistent with the scenario described above.

The VM (Listing 1) complies with the W3C VC data model specification [7], [8], and, consequently, follows the typical VC structure. It includes the following properties: (1) `@contect`: specifies a JSON-LD context that defines the VM's semantics (e.g., W3C VC context and additional domain-specific vocabularies), (2) `type`: describes the type of credential (e.g., VerifiableCredential, VerifiableMandate, DelegatedStudentEnrollment), (3) `issuer`: defines the entity that issued the VM (Alice DID), (4) `issuanceDate`: provides information about the VM's issuance date-time, (5) `validFrom`, and `validUntil`: limits the validity (period) of the VM, (6) other metadata about the VM, such as `id`, `name`, `description`, `termsOfUse`, which enhance the VM by providing additional descriptive information and help to understand the VM's context and use, (7) `credentialSubject`: provides information about the entity that the VM describes, hence, it contains claims about the VM subject. Our use case includes data indicating that Alice has authorized Bob to act on her behalf and enroll her in the Faculty in the selected program.[11] (8) `proof`: contains a digital signature or cryptographic proof(s), and is crucial for maintaining the verifiability, integrity, authenticity, and non-repudiation of the VM. To increase functionality, credibility, traceability, and trust, and to ensure standardization and interoperability, the provided VM includes additional elements, namely, `credentialStatus`, `credentialSchema`, and `evidence`. (9) `credentialStatus`: indicates the VM's status (e.g., revocation, suspension, etc.) and allows

[11] As this property is the most imperial and requires some customization for delegation, we will address it in more detail below.

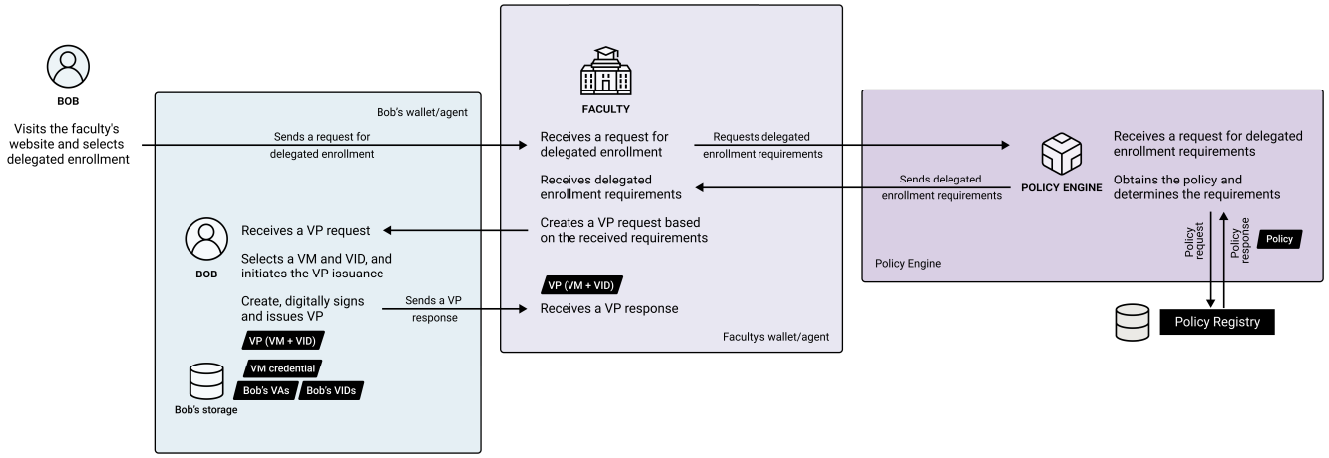


FIGURE 20. Bob creates, signs, and issues a VP and sends it to the Faculty.

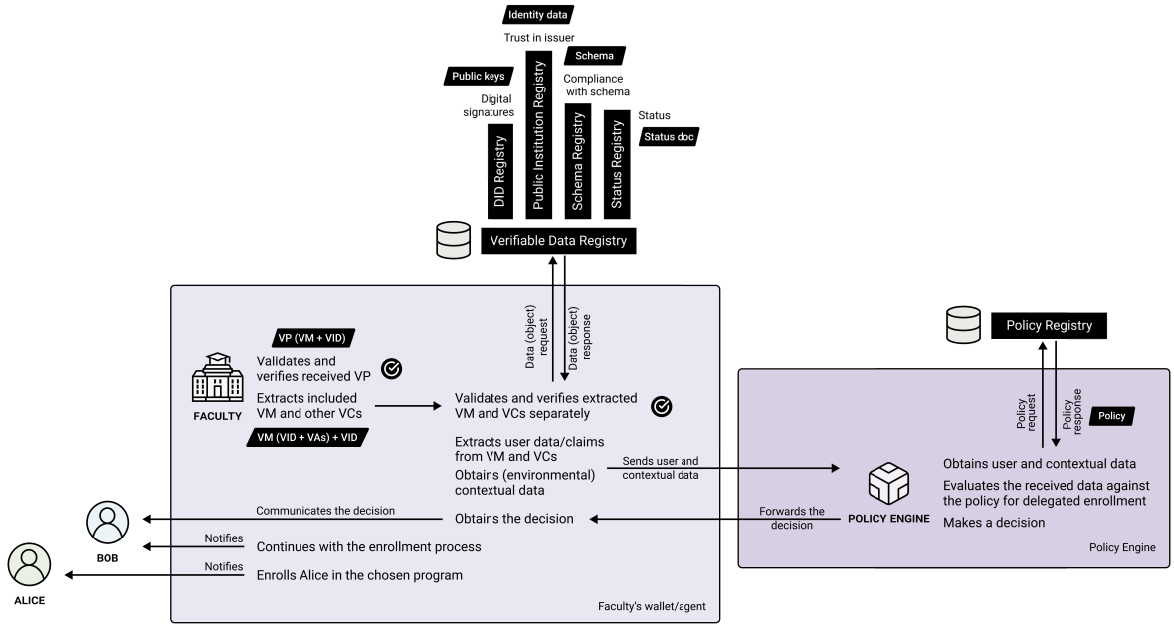


FIGURE 21. The Faculty validates and verifies the VM credential, extracts and evaluates data, and makes a decision regarding delegated enrollment.

the verifiers to verify the VM's validity at the time of presentation, (10) `credentialSchema`: references to the VM schema that defines the VM's structure and allows verifiers to validate the compliance of the VM credential with the schema, (11) `evidence`: provides references or data that supports the claims in the credential, which can be used to increase trust.

The fundamental properties (like `id`, `type`, `issuer`, `proof`, `issuanceDate`, `proof`, etc.) remain consistent with the W3C VC data model specification, which ensures security, interoperability, and standardization of the core VM elements. On the other hand, the `credentialSubject` property requires some customization to support delegation (as demonstrated in Listing 1). It must contain all the information that allows the delegate to act on behalf of the delegator, including: (i) data that identify and describe (Bob), the delegate, (ii) data that identify and describe (Alice), the delegator, (iii) data about authorizations or delegated actions (e.g., enrollment into the Faculty) and their constraints (e.g., regarding the selected Faculty, study program, year of enrollment, electives, and any additional restrictions, such as the time and location of the VM utilization, etc.), (iv) additional credentials that are required to identify the delegator (Alice), and prove the delegator's entitlement/authorization to perform the actions that the delegator wants to delegate further.

Thus, regardless of the use case or domain-specifics, to support delegation, the `credentialSubject` must include the following properties: `id` (delegate) and one or more `permissions`, within which the delegated `roles`

```
{
  "@context": ["https://www.w3.org/2018/credentials/v2",
       "https://schema.org"],
  "id": "https://example.org/credentials/3732",
  "type": ["VerifiableCredential", "VerifiableMandate", "
      DelegatedStudentEnrollment"],
  "name": "Delegated Student Enrollment",
  "description": "Credential verifying that the issuer
       has empowered the subject to enroll him/her into
       the faculty in the desired program.",
  "issuer" : "did:example:1234",
  "issuanceDate": "2024-08-01TT00:00:00Z",
  "validFrom": "2024-09-01TT00:00:00Z",
  "validUntil": "2024-09-30T00:00:00Z",
  "credentialSubject": {//Data about the delegate Bob (VM
       subject/holder) and obtained permissions
    "id": "did:example:5678",
    "name": "Bob Doe",
    "nationalID": {"propertyID": "emso", "value":
        "1204999506788"},
    "email": "bob.doe@example.si",
    "permissions": [{//Permissions and constraints that
        the delegator Alice (VM issuer) transferred to
        the delegate Bob (VM subject/holder)
      "id": "PERM1_enrollment",
      "name": "Enrollment into the faculty",
      "actions": ["Apply", "Enroll", "Submit forms", "
          Provide documentation", "Contact"],
      "role": "family",
      "grantedBy": {//Data about the delegator Alice (
          VM issuer)
        "id": "did:example:1234",
        "name": "Alice Doe",
        "birthDate": "2006-06-26",
        "birthPlace": {"address": {"addressLocality": "
            Maribor", "addressCountry": "SLO"}},
        "address": {"streetAddress": "Smetanova ulica
            12", "addressLocality": "Maribor", "
            postalCode": "2000", "addressCountry": "SLO
            "},
        "nationalID": {"propertyID": "emso", "value":
            "2606006501233"},
        "documentID": {"propertyID": "ID Card", "value
            ": "SI12345678"},
        "phoneNumber": "+386 41 123 456",
        "email": "alice.doe@example.si",
      },
      "constraints": {//Permission constraints and
          details
        "location": "https://faculty.example.com/enroll
            ",
        "validFrom" : "2024-08-01T19:23:24Z",
        "validUntil" : "2024-09-30T00:00:00Z",
        "faculty": {"id": "did:example:faculty", "name
            ": "FERI UM", "address": {...}},
        "studyProgram": {"id": "IDT-176", "name": "
            Informatics and Data Technologies", "
            academicYear": "2024/2025", "cycle": "First
             Cycle (Bachelor's)", "studyYear": "1", "
            electives": [{...}, {...}, {...}]}
      },
      "verifiableCredential": [{...}, {...}, {...}], //
          Additional credentials needed to identify (
          VID) Alice and enroll her in the faculty (VAs
          )
    },{...},{...}
  ],
},
"credentialStatus": {...}, //Reference to the status
     list.
"credentialSchema": {...}, //Reference to the VM schema
     for delegated student enrollment.
"termsOfUse": [...],
"evidence": [{...}, {...}, {...}],
"proof": {...} //Digital, cryptographic proof, issuer's
     (Alice's) signature.
}
```


LISTING 1. An example of a Verifiable Mandate (VM) credential verifying that the issuer, Alice, has empowered the holder, Bob, to enroll her into the Faculty in the desired program.

and `actions` are defined and restricted (`constraints`). At the same time, the encapsulated properties are domain-specific, and should be adjusted to support a specific use case, workflow, or (business) process requirements.

V. DISCUSSION

By utilizing the proposed decentralized, user-centric authority and access delegation model based on the SSI principles, properties, and technologies (DIDs, VCs, VPs), we can satisfy the requirements of IIC and support the delegation within existing use cases, workflows, and (business) processes within companies and organizations. We can increase the SSI model's functionality and flexibility by extending and adapting it for delegation, as presented in Section III and validated in Section IV. Therefore, identity holders can be given even more control over their identity and attributes/claims by allowing them to transfer their powers or capabilities and delegate (part of) their work/tasks/rights/responsibilities to another entity that can represent them in different interactions, which is crucial in various scenarios, such as, e.g., to relieve entities of workload, provide support for cases when entities cannot perform specific tasks themselves for various reasons, increase collaboration and cooperation, reallocate, redistribute or reassign work, and ensure greater efficiency, etc. As the proposed delegation model represents an extension of the SSI, it brings all the advantages of the SSI approach [4], [57], including decentralization and user centrism. Within the SSI ecosystem, end users, i.e., identity holders, retain control over their identifiers and associated identity data/claims/attributes, and control the data flow in digital interactions. To summarize, they can create multiple identifiers independently of external entities, obtain identity data/claims/attributes from various sources, and thus build on their digital identity while being responsible for their safe, autonomous storage, management, and distribution. The model enables digitization and automation of workflows and (business) processes, and brings many other advantages, such as simplicity of the UI and improved UX. Moreover, utilizing decentralized technologies and cryptographic mechanisms can increase security, privacy, trust, and transparency, and achieve data minimization [58]. In the following, we will discuss the advantages mentioned, and link them to a validated case study for a better illustration and understanding.

1) **Digitization:** As we demonstrated in Section IV, the delegated enrollment process, which currently takes place physically at the Faculty in most cases, can be transferred to a digital environment, enabling students and their delegates or representatives to act entirely online. In the SSI ecosystem, all the phases of delegation described above, i.e., issuance (delegator, Alice), utilization (delegate, Bob), and validation and verification, including the decision on delegation (VM verifier, Faculty), can take place electronically, without the need for the physical presence of the involved entities. The Faculty can digitize and automate the entire (delegated) enrollment process, including validation, verification, and decision-making. The delegator (Alice)

can delegate enrollment electronically, by issuing a VM credential and forwarding it to the delegate (Bob), who can initiate the delegated enrollment process and enroll the delegator in the Faculty.

2) **Automation:** The utilization of the proposed delegation model can speed up the workflows and process significantly by automating the process flows and allowing the exchange, processing, validation, and verification of data and exchanged credentials to be fully (or partly) automated, and carried out remotely without the need for physical presence, with a high level of accuracy, validity, and assurance. Moreover, agent automation minimizes the entities' direct involvement, reducing the risk of errors, misuse, and unauthorized access. The direct inclusion of entities is required only in the creation and management of VM schemas and policies (VM verifier, Faculty), in the generation (delegator, Alice) and acceptance of VM credentials, and the initiation of the delegated authority or access, e.g., delegated enrollment (delegate, Bob). On the other hand, the entire VM credentials' validation and verification, as well as decision-making on the part of the VM verifier (Faculty), can be completely automated, which can speed up the delegated enrolment process significantly, and require fewer resources (time, cost, people).
3) **Simplicity and User Experience:** Due to the utilization of wallets/agents and process automation, the delegated enrolment using the proposed delegation model is simple and intuitive for all entities. They can carry out the entire delegated enrollment using SSI digital identity wallets/agents with a few clicks, QR code scans, and existing security mechanisms, as demonstrated in a few examples below. Firstly, by scanning a QR code or clicking on a connection invitation link followed by confirmation, the entities can exchange DIDs, resolve them to DID documents, and establish private and secure peer-to-peer DID connections. Secondly, based on the retrieved VM schema, the delegator's (Alice's) agent generates a form for issuing a VM credential automatically. Therefore, Alice simply enters the required data, selects the required credentials and the recipient (delegate). By confirming the VM credential generation followed by optional authentication, the agent signs the VM credential with her private key and forwards it to the delegate's agent automatically. Thirdly, after retrieving the VP/VM proof request by the VM verifier (Faculty), the delegate's (Bob's) agent can generate the VP automatically. However, if Bob has several credentials in his wallet that match the VP/VM request requirements, the agent suggests which credentials to include, and Bob selects the ones to include in the VP. This is followed by confirmation, optional authentication, VP signing, and the transmission of the VP to the verifier, which is similar to the VM issuance process described above.
4) **Security and Trust:** Security and trust are ensured at several levels in the proposed delegation model through DIDs, VCs (VMs), VDRs, cryptographic mechanisms, and trust frameworks. Communication and exchange of data and credentials/presentations, precisely VM credentials, throughout the entire process, is carried out via secure, trustworthy, private peer-to-peer connections established beforehand by the involved entities (delegator and delegate, delegate and VM verifier). The authenticity, credibility, and integrity of VMs are ensured through digital signature checks, which enable VM verification and immutability or tamper resistance. Moreover, trust is established by utilizing VDR registries, protected by the appropriate AC mechanism. VDRs serve as a SSOT, and enable verification of public, trusted issuers, consequently instilling trust in the VM credential. On the other hand, for building trust in private VM issuers, the proposed model suggests the utilization of the VID credentials issued by trusted issuers that have their DIDs rooted in a public VDR and attest to the VM issuer's identity.
However, VDRs are essential, not only for issuers' identification and authentication (DID Registry, Public Institution Registry), but also for other aspects of VM validation and verification, such as checking compliance with the VM schema (Schema Registry), checking the VM's validity (Status Registry), and for auditing and timestamping purposes (Blockchain Anchor). The VM verifier (Faculty) can validate and verify the VP, including the VM and other included credentials, by utilizing the VM Credential Validator and performing multiple checks as described in Section III-D4. Therefore, checking (i) compliance with the VM schema, (ii) validity period, (iii) VM status, (iv) digital signatures, and (v) issuer's identity.
5) **Privacy and Data Minimization:** The novel delegation model utilizing the VM credential, together with cryptographic mechanisms such as selective disclosure (partial data sharing, e.g., BBS+ signatures, CL signatures) and ZKP (privacy-preserving proofs, e.g., zk-SNARKs, zk-STARKs, Bulletproofs), can help increase privacy significantly, and minimize the transmission and disclosure of data, as they enable delegators and delegates to reveal only attributes, or even predicates, about those attributes to the VM verifiers. In the current delegated enrollment process, in addition to the authorization form, the delegate must also deliver a copy or a scan of the delegator's identification document to the verifier. Moreover, the delegate must obtain all the required documentation for enrollment from the delegator (e.g., a certificate of payment of the enrollment fee, a certificate of completed high school, a certificate of graduation, etc.), which brings privacy concerns, and does not prevent the delegate from using the obtained documents independently and for any purpose other than for the enrollment of the delegator. We can reduce the mentioned concerns by utilizing VMs with the concept of changing. In addition to defining the

TABLE 5. Advantages and disadvantages of delegation utilizing the proposed model compared to the current process, i.e., without utilizing the model.

	Current delegation process without the model	Delegation process with the model
Digitization	1. Not digitalized or partially digitalized. 2. Requires the physical presence of the involved entities (time-consuming). 3. Gathering documentation manually, making a copy/scan of the ID, preparing and signing an authorization form. Transferring documents manually or electronically between the delegator and the delegate. Physical presentation of documents to the verifier and manual verification. 4. No initial setup investments regarding time, effort, and cost, but a burden for the verifier during the validation and verification phase.	1. Fully digitalized. 2. Physical presence is not required. 3. Supports for various real-world use cases in physical and digital settings. 4. Digital credential creation, collection, storage, and transfer are enabled by a digital identity wallet/agent. 5. It requires the establishment of infrastructure, as well as schema and policy definition. The initial investment regarding time, cost, and effort is higher for verifiers (delegation enablers). 6. The verification phase is improved, streamlined, sped up significantly, and requires no human intervention.
Automation	1. The absence of digitalization hinders automation. 2. Active participation of the involved entities is required. 3. The burden for the verifier. 4. Increased workload, mental fatigue, and a higher likelihood of human errors. 5. Demands significant time and resources.	1. Digitalization facilitates automation. 2. Utilization of SSI wallets/agents. 3. Automation speeds up the process execution significantly and reduces the processing costs. 4. Agents can automate the workflow fully or partially. 5. Minimal (direct) human involvement is required. 6. Validation, verification, and decision-making can be automated entirely from the verifiers' perspective, effectively reducing their workload. 7. The possibility of (human) errors, abuse, and unauthorized access is reduced.
User Experience	1. Inconsistent UX. 2. Lack of a clearly defined delegation procedure. 3. Variability in delegation credentials (usually in paper form/document.) 4. Inconsistencies within and across domains. 5. It requires many steps, physical presence, manual work, and time and resources. 6. Mobile phones and technological knowledge are not required.	1. Unified UX within and across domains. 2. Clearly defined, thus predictable delegation procedure. 3. Clearly defined means, i.e., delegation credentials. 4. Simple, intuitive, and quick. 5. Minimal effort for the involved entities. 6. It does not require a physical presence; it can be done remotely. 7. Leveraging digital identity wallets/agents (mobile first-design, familiar UX). 8. Validation, verification, and decision-making are entirely automated. Thus, no manual work is required. 9. Anticipated challenges, as SSI is a relatively new approach. 10. Learning curve for the involved entities.
Security & Trust	1. Depending on the authenticity of the documents. 2. The possibility of forgery. 3. Questionable trust, authenticity and integrity of the authorization form.	1. Ensured at several levels (DIDs, VCs, VDRs, cryptographic mechanisms, and trust frameworks). 2. Communication and data exchange via secure, trustworthy, and private peer-to-peer DID connections. 3. Different identifiers can be used or different interactions (minimizing correlation). 4. Digital signatures and their verification ensure credentials' authenticity, credibility, and integrity, ensuring immutability or tamper resistance. 5. Trust is enabled by utilizing VDR registries, serving as a SSOT, establishing trust in entities and credentials (proofs), and enabling their validation and verification.
Privacy & Minimization	1. Full credentials are exchanged. 2. Not only a minimal amount of data shared with the delegates and verifier. 3. If the delegator's original documents are required, the delegate has direct access to all the documents (possible misuse or abuse). 4. Posing privacy concerns. 5. Delegators do not remain in charge of their credentials.	1. Significantly increased privacy and data minimization (when issuing VCs and presenting VPs). 2. Utilizing VCs and cryptographic mechanisms. 3. Exchanging only a selective set of attributes or predicates, thus only the minimum amount of data are necessary for a certain interaction. 4. No need for full credentials' exchange. 5. Proofs from different credentials can be chained within delegation credentials (increased complexity, but improved security and privacy). 6. Delegators remain in charge of their credentials.

scope of the delegation (permissions, constraints, etc.) in the VM, the delegator (Alice) can embed/include additional credentials (VID, VAs), thereby preventing their independent use, potential misuse or abuse, and

transfer to third parties. Chaining credentials within the VM increases its complexity. Thus, the validation and verification of the VM are becoming more complex, but, at the expense of the chaining, security and privacy can be increased significantly. By using mechanisms such as selective data disclosure and ZKP, we can increase privacy further and minimize the data presented, both when issuing a VM credential and when generating a VP, and enable the transmission of the minimum amount of data necessary for a certain interaction. The delegator (Alice) thus maintains control over her data/credentials, and includes only those (data/credentials) necessary for enrollment.

Additionally, Table 5 summarizes the advantages and disadvantages of delegation utilizing the proposed model compared to the existing process without the model utilization.

In addition to all the mentioned advantages of the novel authority and access delegation model that we have highlighted and discussed above, we must not forget all the inherited advantages and benefits arising from using SSI approaches and technologies [4].

VI. CONCLUSION

Digital identification, authentication, and authorization are important aspects of ensuring the security and reliability of digital interactions. By utilizing the SSI approach, we can simplify these processes and enable users, i.e., identity holders, to gain control over their digital identity and data. SSI brings many advantages, and reduces the shortcomings of the traditional, centralized Identity and Access Management (IAM) system that we highlighted in the introductory sections (Section I - II), e.g., centralization, intermediaries, SPOF, identity abuse and theft, security and privacy issues, etc. It represents a significant leap in the field of IAM, and it brings many advantages, such as decentralization, control, user-centricity, improved security, privacy, sovereignty, data minimization, selective disclosure, interoperability, and mobile-first design.

However, despite all its positive features, SSI is focused on direct interactions between two independent entities. Hence, it supports direct identification and access to resources and services, where the identity holder is the authenticating bearer of his/her credentials. In this regard, SSI does not address primarily IIC cases, such as guardianship, controllership, and delegation, which are key parts of life situations and natural human relationships in the physical world. As a result, SSI fundamentally excludes individuals who are unable to act independently in the digital world due to various social, legal, mental, physical, or other limitations, e.g., children, the elderly, homeless, animals, things, etc. Additionally, it does not address scenarios that require delegation of work, tasks, rights, responsibilities, capabilities, etc. By default, it does not support authority or access delegation, arising from the need for greater efficiency, relief, or distribution of work, collaboration, cooperation, the inability to perform work, tasks, rights, responsibilities for various reasons, etc. Therefore, SSI focuses on direct interactions, while the use cases requiring indirect control are extremely limited and require additional research efforts.

Consequently, our research aimed to reduce the aforementioned gaps and address the delegation process comprehensively utilizing SSI principles, properties, and technologies. Thus, we present a novel authority and access delegation model utilizing VCs (Section III). It represents an extension of the SSI, and enables the delegation of existing use cases, workflows, and (business) processes of organizations and companies that provide resources and services to end users, i.e., identity holders. The paper provides a comprehensive model presentation, which includes all the basic building blocks (Section III-A) and data flows (Section III-B), a presentation of the delegation phases, processes, and sub-processes (Section III-C), identification and definition of technology-independent components (Section III-D) and their utilization from the perspective of different entities (Section III-E). We validated the proposed novel model using a case study methodology (Section IV), which will be expanded in further research by including more representative case studies from different domains, allowing us to assess validity and robustness in different contexts.

A. LIMITATIONS AND FUTURE WORK

Although the proposed model brings many advantages (discussed in Section V) and extends the utilization of the SSI approach by enabling indirect identity control, particularly through delegation of authority or access, it still has certain limitations that can be addressed in future work. In addition, due to the complexity and characteristics of delegation, plenty of research possibilities could be explored further. Hence, below, the limitations of the proposed delegation model will be presented and organized by category, and additional opportunities will be discussed for further research.

1) **Verifiable Credentials:** The model is limited to the SSI ecosystem, leveraging its core principles, properties, and technologies. As mentioned, within the SSI, we can utilize one of three possible approaches, namely, delegation with (1) DIDs and DID documents, (2) VCs, and (3) shared digital identity wallets. We focused on the second approach, utilizing VCs, which ensures the authenticity of information, provides flexibility and granularity, and allows delegators to determine the scope and limitations of delegated roles, attributes, rights, permissions, actions, or claims (in general) dynamically, of course, within the framework set by the organization or company that enables the delegation. Although the model requires VCs and is limited in that regard, it is important to emphasize that it is designed to be technology-agnostic regarding identification, authentication, and communication mechanisms. Even though the model focuses on utilizing VCs in combination with DIDs, it can remain unchanged, even if classic eID is

used for identification and authentication purposes, as is the case with the EU Digital Identity Wallet (EUDIW). Therefore, DIDs may be replaced by alternative mechanisms, as long as they fulfill the necessary functional requirements, which will be explored and validated further in our subsequent research efforts.

2) **Identity wallets/agents:** All the involved entities must have digital identity wallets/agents to participate in the delegation process. Consequently, the model is inherently limited to entities with access to smart devices. This limitation stems from the nature of digitalization, and is also a broader limitation of the SSI paradigm. A potential solution to this issue lies in introducing digital guardians, a concept already explored in initiatives such as those by the Sovrin Foundation [22].
3) **Organizational context:** The model is limited to supporting organizational processes, use cases, and workflows. In the initial stages, an appropriate infrastructure (e.g., VDR registries, Policy Engine, etc.) must be established, and credential schemas and delegation policies must be defined. These elements are essential for issuing and verifying VM credentials, as well as facilitating decision-making processes regarding the approval or denial of delegated authority and access. Nevertheless, the model could potentially be adapted to support other delegation scenarios outside of the organizational context. Such adaptations, however, require validation and represent additional opportunities for future research.
4) **Identity holders:** Although the model supports extending existing the processes, use cases, and workflows within organizations—who typically act as credential issuers or verifiers—its primary emphasis is on delegation initiated by identity holders. Therefore, identity holders become the issuers of delegation credentials, allowing them to delegate authority or access rights to other identity holders, who can, subsequently, act on their behalf in interactions with verifiers. Consequently, the approach is fundamentally user-centric, and excludes delegation scenarios that involve delegating credential issuance or verification capabilities to other entities. VCs could be combined with VDRs to facilitate delegation by issuers and verifiers, as the issuers and verifiers are typically public legal entities. Parallels to delegation by an issuer can be observed in the work conducted by EBSI [59], particularly through its introduction of the Issuer Trust Model. This model defines the mechanisms by which trust is conveyed between issuers across different levels of authority, achieved through the issuance of Verifiable Accreditations stored publicly in the Trusted Issuers Registry.
5) **Human-to-human:** The model focuses primarily on supporting real-life scenarios arising from natural human interactions involving the exchange of delegation credentials between human entities. While we anticipate it could also support other interaction types, such as human-to-machine interactions, this capability has not yet been validated. Thus, exploring and validating the model's applicability to additional interaction scenarios presents a valuable avenue for future research.
6) **Direct delegation:** The model aims to support direct delegation between two identity holders, namely, a delegator and a delegate, thus, not focusing on sub-delegation involving multiple intermediaries. Sub-delegation represents a more complex use case of transferring delegated authority or access from one entity to another. It requires additional considerations related to the VM and schema structure (e.g., limiting the delegation chain, etc.), trust chain management and verification, auditing, revocation, etc. These are all essential for effectively supporting various delegation use cases, both complex cases involving sub-delegation and intermediaries, and more straightforward cases involving merely the transfer of authority or access between the delegator and delegate. Since the proposed model is constrained to supporting direct delegation, future research could examine the adjustments necessary to accommodate more complex delegation scenarios involving intermediaries.
7) **VM revocation and suspension:** Revocation and suspension of VM credentials are also crucial, and represent one of the delegation requirements. It allows delegators to maintain control over their identity and data, preventing potential abuses and misuse of delegated powers, especially in changed circumstances. Although the proposed model assumes the utilization of one of the existing revocation mechanisms, it would be necessary to verify which mechanism is the most appropriate for various delegation requirements that stem from the delegation characteristics. Thus, different scenarios should be addressed, such as revocation by (i) the delegator (VM issuer), (ii) the VM verifier upon detected errors, abuses or misuses, or when specified within the VM by the delegator, (iii) intermediaries in the delegation chain (sub-delegation), as well as automated revocation after (iv) the VM expiration, or (v) successful execution of the delegated authority or access (e.g., when the VM is limited to single use), etc.

As emphasized several times throughout the paper, delegation and mandates represent an important part of real-life scenarios we often encounter in physical and digital environments. However, delegation within the SSI ecosystem remains relatively unexplored, but offers interesting research opportunities. Many unsolved challenges in this context require innovative solutions that will enable secure and efficient delegation of authority and access in different contexts and domains. Therefore, research and development in this area is essential for the wider adoption and use of SSI technologies and approaches. Only in this way will we achieve a reliable and user-friendly IdM, which will contribute to greater security, privacy, and trust in digital services.

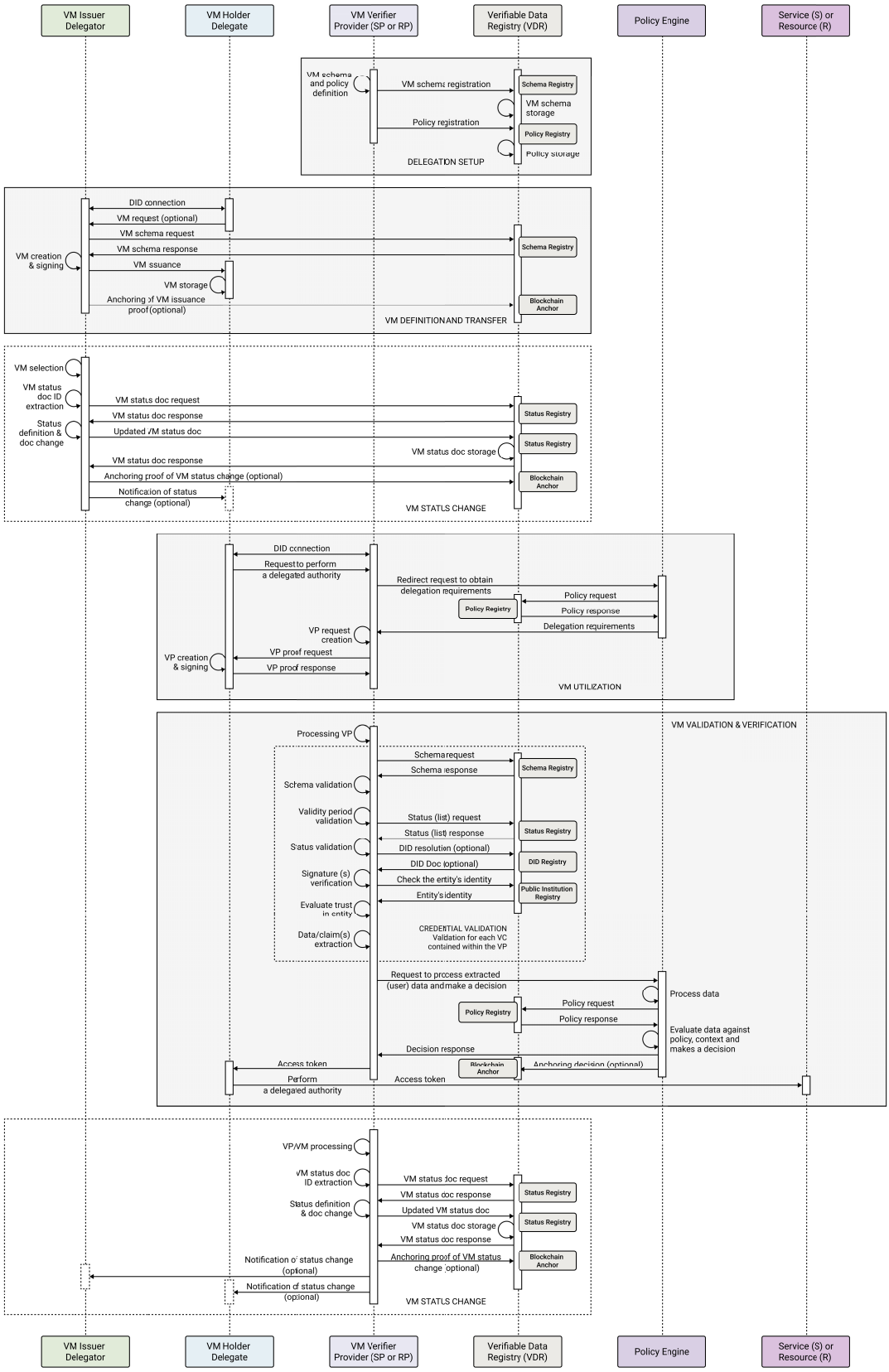


FIGURE 22. Delegation process.

CONFLICT OF INTEREST

The authors declare no conflict of interest. The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

APPENDIX

DELEGATION PROCESS DETAILED VISUALIZATION

Fig. 22 provides a comprehensive overview of the entire delegation process, including all stages and their details.

REFERENCES

[1] Q. Pham, J. Reid, A. McCullagh, and E. Dawson, "On a taxonomy of delegation," *Comput. Secur.*, vol. 29, no. 5, pp. 565–579, Jul. 2010. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167404809001473

[2] A. Ali, U. Habiba, and M. A. Shibli, "Taxonomy of delegation model," in *Proc. 12th Int. Conf. Inf. Technol. - New Generat.*, Apr. 2015, pp. 218–223. [Online]. Available: https://ieeexplore.ieee.org/document/7113476

[3] G. Yin, D. Shi, M. Guo, and H. Wang, "SRDL: A simple role-based delegation logic," in *IEEE MTT-S Int. Microw. Symp. Dig.*, Aug. 2009, pp. 913–916. [Online]. Available: https://ieeexplore.ieee.org/document/5331636

[4] Š. Čučko, Š. Bećirović, A. Kamišalić, S. Mrdovic, and M. Turkanović, "Towards the classification of self-sovereign identity properties," *IEEE Access*, vol. 10, pp. 88306–88329, 2022. [Online]. Available: https://ieeexplore.ieee.org/document/9858139

[5] E. Commission. (2024). *A Digital Id and Personal Digital Wallet for Eu Citizens, Residents and Businesses*. [Online]. Available: https://ec.europa.eu/digital-building-blocks/sites/display/EUDIGITALIDENTITYWALLET

[6] M. Sporny, D. Longley, M. Sabadello, D. Reed, O. Steele, and C. Allen. (2022). *Decentralized Identifiers (DIDS) V1.0*. [Online]. Available: https://www.w3.org/TR/did-core

[7] M. Sporny, D. Longley, and D. Chadwick. (2022). *Verifiable Credentials Data Model 1.1*. [Online]. Available: https://www.w3.org/TR/vc-data-model-1.1

[8] M. Sporny, D. Longley, D. Chadwick, and O. Steele. (2024). *Verifiable Credentials Data Model 2.0*. [Online]. Available: https://www.w3.org/TR/vc-data-model-2.0

[9] A. Abraham, S. More, C. Rabensteiner, and F. Hörandner, "Revocable and offline-verifiable self-sovereign identities," in *Proc. IEEE 19th Int. Conf. Trust, Secur. Privacy Comput. Commun. (TrustCom)*, Dec. 2020, pp. 1020–1027. [Online]. Available: https://ieeexplore.ieee.org/document/9343191

[10] A. Abraham, K. Koch, S. More, S. Ramacher, and M. Stopar, "Privacy-preserving eID derivation to self-sovereign identity systems with offline revocation," in *Proc. IEEE 20th Int. Conf. Trust, Secur. Privacy Comput. Commun. (TrustCom)*, Oct. 2021, pp. 506–513.

[11] R. Chotkan, J. Decouchant, and J. Pouwelse, "Distributed attestation revocation in self-sovereign identity," in *Proc. IEEE 47th Conf. Local Comput. Netw. (LCN)*, Aug. 2022, pp. 414–421. [Online]. Available: https://ieeexplore.ieee.org/document/9843323

[12] D. Schumm, R. Mukta, and H.-Y. Paik, "Efficient credential revocation using cryptographic accumulators," in *Proc. IEEE Int. Conf. Blockchain Cryptocurrency (ICBC)*, May 2023, pp. 1–3. [Online]. Available: https://ieeexplore.ieee.org/document/10237019

[13] P. Manimaran, A. F. Da Conceição, T. Garrett, M. Raikwar, and R. Vitenberg, "Prevoke: Privacy-preserving configurable method for revoking verifiable credentials," in *Proc. IEEE Int. Conf. Blockchain (Blockchain)*, Aug. 2024, pp. 354–361.

[14] A. M. Papathanasiou and G. C. Polyzos, "Privacy-preserving revocation of verifiable credentials with verifiable random functions," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, Jan. 2024, pp. 391–394. [Online]. Available: https://ieeexplore.ieee.org/document/10572151

[15] N. Sitouah, F. Bruschi, F. L. Pallotta, R. Mencucci, and D. Sciuto, "An untraceable credential revocation approach based on a novel Merkle tree accumulator," in *Proc. IEEE Int. Conf. Blockchain Cryptocurrency (ICBC)*, May 2024, pp. 210–214. [Online]. Available: https://ieeexplore.ieee.org/document/10634387

[16] A. Mühle, A. Grüner, T. Gayvoronskaya, and C. Meinel, "A survey on essential components of a self-sovereign identity," *Comput. Sci. Rev.*, vol. 30, pp. 80–86, Nov. 2018. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1574013718301217

[17] Y. Bai, H. Lei, S. Li, H. Gao, J. Li, and L. Li, "Decentralized and self-sovereign identity in the era of blockchain: A survey," in *Proc. IEEE Int. Conf. Blockchain*, Aug. 2022, pp. 500–507.

[18] P. Dunphy and F. A. P. Petitcolas, "A first look at identity management schemes on the blockchain," *IEEE Secur. Privacy*, vol. 16, no. 4, pp. 20–29, Jul. 2018. [Online]. Available: https://ieeexplore.ieee.org/document/8406563

[19] O. Avellaneda, A. Bachmann, A. Barbir, J. Brenan, P. Dingle, K. H. Duffy, E. Maler, D. Reed, and M. Sporny, "Decentralized identity: Where did it come from and where is it going?" *IEEE Commun. Standards Mag.*, vol. 3, no. 4, pp. 10–13, Dec. 2019. [Online]. Available: https://ieeexplore.ieee.org/document/9031542

[20] K. Wagner, B. Némethi, E. Renieris, P. Lang, E. Brunet, and E. Holst, "Self-sovereign identity: A position paper on blockchain enabled identity and the road ahead," Identity Working Group German, Blockchain Association (Blockchain Bundesverband), Berlin, Germany, Tech. Rep., Oct. 2018.

[21] D. Hardman. (2019). *Aries RFC 0103: Indirect Identity Control*. [Online]. Available: https://github.com/hyperledger/aries-rfcs/tree/main/concepts/0103-indirect-identity-control

[22] S. G. T. Force. (2019). *On Guardianship in Self-sovereign Identity V1.2*. [Online]. Available: https://sovrin.org/wp-content/uploads/Guardianship-Whitepaper2.pdf

[23] G. Fedrecheski, J. M. Rabaey, L. C. P. Costa, P. C. Calcina-Ccori, W. T. Pereira, and M. K. Zuffo, "Self-sovereign identity for IoT environments: A perspective," in *Proc. Global Internet Things Summit (GIoTS)*, Jun. 2020, pp. 1–6.

[24] D. Hardman. (2019). *Delegation Details*. [Online]. Available: https://github.com/hyperledger/aries-rfcs/blob/main/concepts/0103-indirect-identity-control/delegation-details.md

[25] D. Hardman and L. Harchandani. (2019). *Aries RFC 0104: Chained Credentials*. [Online]. Available: https://github.com/hyperledger/aries-rfcs/blob/main/concepts/0104-chained-credentials/README.md

[26] S. Foundation. (2019). *Sovrin Glossary V3*. [Online]. Available: https://sovrin.org/wp-content/uploads/Sovrin-Glossary-V3.pdf

[27] *Verifiable Credentials Lifecycle 1.0*, D. Chadwick, W3C, MA, USA, Dec. 2020. [Online]. Available: https://w3c-ccg.github.io/vc-lifecycle/

[28] M. Sporny, D. Longley, and D. Chadwick. (2022). *Subject-holder Relationships*. [Online]. Available: https://www.w3.org/TR/vc-data-model/#subject-holder-relationships

[29] S. Lim, M.-H. Rhie, D. Hwang, and K.-H. Kim, "A subject-centric credential management method based on the verifiable credentials," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, Jan. 2021, pp. 508–510.

[30] S. Foundation. (2024). *Guardianship*. [Online]. Available: https://sovrin.org/guardianship

[31] S. G. T. Force. (2023). *On Guardianship in Self-sovereign Identity V2*. [Online]. Available: https://sovrin.org/wp-content/uploads/Guardianship-Whitepaper-V2.0.pdf

[32] T. O. I. Foundation. (2022). *The Toip Model*. [Online]. Available: https://trustoverip.org/toip-model

[33] T. O. I. Foundation. (2021). *Introduction to Trust Over Ip*. [Online]. Available: https://trustoverip.org/wp-content/uploads/Introduction-to-ToIP-V2.0-2021-11-17.pdf

[34] M. Davie, D. Gisolfi, D. Hardman, J. Jordan, D. O'Donnell, D. Reed, and O. van Deventer. (2019). *0289: The Trust Over Ip Stack*. [Online]. Available: https://github.com/hyperledger/aries-rfcs/tree/main/concepts/0289-toip-stack

[35] S. G. W. Group. (2021). *Guardianship Credentials Technical Requirements*. [Online]. Available: https://drive.google.com/file/d/1M21PznPAd0H6z1t4ODl-jiEoXZjEhwcb/view

[36] S. G. W. Group. (2021). *Guardianship Credentials Implementation Guidelines*. [Online]. Available: https://drive.google.com/file/d/1vBePVx8n3MRDWcePkwVDya9ab4BHEyU/view

[37] D. Hardman. (2019). *Sample Guardianship Schema*. [Online]. Available: https://github.com/hyperledger/aries-rfcs/blob/main/concepts/0103-indirect-identity-control/guardianship-sample/schema.md

[38] D. Hardman. (2019). *Sample Guardianship Trust Framework*. [Online]. Available: https://github.com/hyperledger/aries-rfcs/blob/main/concepts/0103-indirect-identity-control/guardianship-sample/trust-framework.md

[39] Y. Dündar and I. Sertkaya, "Self sovereign identity based mutual guardianship," *J. Modern Technol. Eng.*, vol. 5, no. 3, pp. 189–211, 2020.

[40] R. Mukta, H.-Y. Paik, Q. Lu, and S. S. Kanhere, "Credential-based trust management in self sovereign identity," in *Proc. Womencourage*, Prague, Czech Republic, Sep. 2021.
[41] R. Mukta, R. C. Teh, H.-Y. Paik, Q. Lu, and S. S. Kanhere, "VCTP: A verifiable credential-based trust propagation protocol for personal issuers in self-sovereign identity platforms," in *Proc. IEEE Int. Conf. Decentralized Appl. Infrastructures (DAPPS)*, Jul. 2023, pp. 109–118.
[42] S. Terzi, C. Savvaidis, A. Sersemis, K. Votis, and D. Tzovaras, "Decentralizing identity management and vehicle rights delegation through self-sovereign identities and blockchain," in *Proc. IEEE 45th Annu. Comput., Softw., Appl. Conf. (COMPSAC)*, Jul. 2021, pp. 1217–1223.
[43] F. Buccafurri, V. De Angelis, and R. Nardone, "How can the holder trust the verifier? A CP-ABPRE-based solution to control the access to claims in a self-sovereign-identity scenario," *Blockchain, Res. Appl.*, vol. 5, no. 3, Sep. 2024, Art. no. 100196. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2096720924000095
[44] E. Commission. (2024). *Ebsi Glossary*. [Online]. Available: https://ec.europa.eu/digital-building-blocks/sites/display/EBSI/Glossary
[45] Š. Čučko, V. Keršič, and M. Turkanović, "Towards a catalogue of self-sovereign identity design patterns," *Appl. Sci.*, vol. 13, no. 9, pp. 1–23, 2023. [Online]. Available: https://www.mdpi.com/2076-3417/13/9/5395
[46] G. Cohen. (2024). *Verifiable Credentials Json Schema Specification*. [Online]. Available: https://www.w3.org/TR/vc-json-schema
[47] R. Bochnia, D. Richter, and J. Anke, "Self-sovereign identity for organizations: Requirements for enterprise software," *IEEE Access*, vol. 12, pp. 7637–7660, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10379079
[48] B. Podgorelec, L. Alber, and T. Zefferer, "What is a (digital) identity wallet? A systematic literature review," in *Proc. IEEE 46th Annu. Comput., Softw., Appl. Conf. (COMPSAC)*, Jun. 2022, pp. 809–818. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/9842532
[49] A. Preukschat and D. Reed, *Self-Sovereign Identity Decentralized Digital Identity and Verifiable Credentials*. Shelter Island, NY, USA: Manning, 2021.
[50] Y. Liu, Q. Lu, H.-Y. Paik, X. Xu, S. Chen, and L. Zhu, "Design pattern as a service for blockchain-based self-sovereign identity," *IEEE Softw.*, vol. 37, no. 5, pp. 30–36, Sep. 2020. [Online]. Available: https://ieeexplore.ieee.org/document/9089840
[51] J. B. Bernabe, J. L. Canovas, J. L. Hernandez-Ramos, R. T. Moreno, and A. Skarmeta, "Privacy-preserving solutions for blockchain: Review and challenges," *IEEE Access*, vol. 7, pp. 164908–164940, 2019. [Online]. Available: https://ieeexplore.ieee.org/document/8888155
[52] Y. Liu, D. He, M. S. Obaidat, N. Kumar, M. K. Khan, and K. R. Choo, "Blockchain-based identity management systems: A review," *J. Netw. Comput. Appl.*, vol. 166, Jun. 2020, Art. no. 102731. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1084804520302058
[53] M. Davie, D. Gisolfi, D. Hardman, J. Jordan, D. O'Donnell, and D. Reed, "The trust over IP stack," *IEEE Commun. Standards Mag.*, vol. 3, no. 4, pp. 46–51, Dec. 2019. [Online]. Available: https://ieeexplore.ieee.org/document/9031548
[54] U. Maribor. *VPIS Na Podiplomski Tudij*. University of Maribor (UM). Accessed: Sep. 5, 2024. [Online]. Available: https://moja.um.si/bodoci-studentje/Strani/vpis-podiplomski.aspx
[55] S. Ministry Higher Educ. Innov. *Prijava Za Vpis Na Visokoolski Tudij*. Accessed: Sep. 5, 2024. [Online]. Available: https://portal.evs.gov.si
[56] CMEPIUS. *Study in Slovenia*. Accessed: Sep. 5, 2024. [Online]. Available: https://studyinslovenia.si
[57] G. Laatikainen, T. Kolehmainen, and P. Abrahamsson, "Self-sovereign identity ecosystems: Benefits and challenges," in *Proc. 12th Scandin. Conf. Inf. Syst.*, vol. 10, 2021, pp. 1–16. [Online]. Available: https://aisel.aisnet.org/scis2021/10
[58] Š. Čučko and M. Turkanović, "Decentralized and self-sovereign identity: Systematic mapping study," *IEEE Access*, vol. 9, pp. 139009–139027, 2021. [Online]. Available: https://ieeexplore.ieee.org/document/9558805
[59] EBSI. (2024). *Issuer Trust Model*. [Online]. Available: https://hub.ebsi.eu/vc-framework/trust-model/issuer-trust-model-v3



ŠPELA ČUČKO received the master's degree in informatics and technology of communication from the Faculty of Electrical Engineering and Computer Science, University of Maribor, Slovenia, where she is currently pursuing the Ph.D. degree in computer science. She is also a Teaching Assistant and a Researcher with the Institute of Informatics and a Research and Development Group Member named Blockchain Lab:UM. Her research interests include decentralized and self-sovereign identities, blockchain technology, usability, and user experience. She is also focusing on authority and access delegation, utilizing SSI and VCs.



MUHAMED TURKANOVIĆ is currently an Associate Professor with the Institute of Informatics, Faculty of Electrical Engineering and Computer Science, University of Maribor (UM), Slovenia, where he is also the Head of Research and Development at the Blockchain Lab:UM, and the Head of the Slovene EDIH DIGI-SI, UM's Coordinator of the H2020 Project DE4A—Digital Europe for All. He was the Managing Director and the CTO of an IT company, from 2013 to 2016. His current research interests include advanced database technologies, cryptography, digital identities, and blockchain.